

# Оглавление

|                                                           |              |
|-----------------------------------------------------------|--------------|
| <b>Введение</b>                                           | <b>iv</b>    |
| Как устроена современная электроника . . . . .            | iv           |
| Из чего состоит компьютер . . . . .                       | iv           |
| Для чего эта книга и как ей пользоваться . . . . .        | vi           |
| <br><b>I Собираем релейный калькулятор</b>                | <br><b>1</b> |
| <b>1 Двоичная система счисления</b>                       | <b>2</b>     |
| 1.1 Перевод числа из двоичной системы счисления . . . . . | 2            |
| 1.2 Перевод числа в двоичную систему счисления . . . . .  | 3            |
| 1.3 Двоичная логика . . . . .                             | 4            |
| 1.4 Задачи . . . . .                                      | 6            |
| <b>2 Релейный конструктор</b>                             | <b>7</b>     |
| 2.1 Выключатели и двоичная логика . . . . .               | 7            |
| 2.2 Электромагнитное реле . . . . .                       | 8            |
| 2.3 Модули . . . . .                                      | 9            |
| 2.4 Соединительные разъёмы . . . . .                      | 10           |
| 2.5 Модуль переключателей . . . . .                       | 10           |
| 2.6 Практикум . . . . .                                   | 11           |
| 2.7 Задачи . . . . .                                      | 13           |
| <b>3 Хранение и передача данных</b>                       | <b>14</b>    |
| 3.1 Ячейка памяти . . . . .                               | 14           |
| 3.2 RS-триггер . . . . .                                  | 14           |
| 3.3 Шина . . . . .                                        | 15           |
| 3.4 Регистр . . . . .                                     | 16           |
| 3.5 Регистровый модуль . . . . .                          | 16           |
| 3.5.1 Практикум . . . . .                                 | 18           |
| 3.5.2 Задачи . . . . .                                    | 19           |
| <b>4 Хранение больше четырёх бит</b>                      | <b>20</b>    |
| 4.1 Регистровый файл . . . . .                            | 20           |
| 4.1.1 Практикум . . . . .                                 | 20           |
| 4.1.2 Задачи . . . . .                                    | 21           |
| 4.2 Восьмибитные регистры . . . . .                       | 22           |
| 4.2.1 Практикум . . . . .                                 | 22           |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| <b>5</b>  | <b>Унарные логические операции</b>                                 | <b>23</b> |
| 5.1       | Отрицание . . . . .                                                | 23        |
| 5.2       | Копирование . . . . .                                              | 23        |
| 5.3       | Сдвиг . . . . .                                                    | 23        |
| 5.4       | Модуль унарных логических операций . . . . .                       | 24        |
| 5.5       | Практикум . . . . .                                                | 25        |
| 5.6       | Задачи . . . . .                                                   | 26        |
| <b>6</b>  | <b>Бинарные логические операции</b>                                | <b>27</b> |
| 6.1       | Логическое И . . . . .                                             | 27        |
| 6.2       | Логическое ИЛИ . . . . .                                           | 27        |
| 6.3       | Исключающее ИЛИ . . . . .                                          | 28        |
| 6.4       | Модуль бинарных логических операций . . . . .                      | 29        |
| 6.5       | Практикум . . . . .                                                | 30        |
| 6.5.1     | Работа модуля . . . . .                                            | 30        |
| 6.6       | Задачи . . . . .                                                   | 31        |
| <b>7</b>  | <b>Коммутация входов и выходов с помощью кросс-модуля</b>          | <b>32</b> |
| 7.1       | Кросс-модуль . . . . .                                             | 32        |
| 7.2       | Практикум . . . . .                                                | 32        |
| <b>8</b>  | <b>Арифметические операции. Сложение</b>                           | <b>36</b> |
| 8.1       | Как складывать двоичные числа . . . . .                            | 36        |
| 8.2       | Сумматор . . . . .                                                 | 37        |
| 8.3       | Модуль сумматора . . . . .                                         | 38        |
| 8.4       | Практикум . . . . .                                                | 39        |
| 8.4.1     | Работа модуля . . . . .                                            | 39        |
| 8.5       | Задачи . . . . .                                                   | 40        |
| 8.5.1     | Восьмибитный сумматор . . . . .                                    | 40        |
| 8.5.2     | Практикум . . . . .                                                | 40        |
| <b>9</b>  | <b>Арифметические операции. Вычитание</b>                          | <b>42</b> |
| 9.1       | Вычитание двоичных чисел . . . . .                                 | 42        |
| 9.2       | Практикум . . . . .                                                | 43        |
| 9.2.1     | Вычитание через сложение с дополнительным обратным кодом . . . . . | 43        |
| 9.2.2     | Вычитание с помощью сумматора и инвертора . . . . .                | 43        |
| <b>10</b> | <b>Умножение и деление</b>                                         | <b>45</b> |
| 10.1      | Умножение двух двухбитных чисел . . . . .                          | 45        |
| 10.2      | Калькулятор для пошагового деления . . . . .                       | 46        |
| 10.3      | Задачи . . . . .                                                   | 46        |
| <b>11</b> | <b>Калькулятор</b>                                                 | <b>47</b> |
| 11.1      | Калькулятор для 7 операций . . . . .                               | 47        |
| 11.2      | Практикум . . . . .                                                | 47        |
| <b>12</b> | <b>Ответы на задачи</b>                                            | <b>49</b> |

|            |                                                                    |           |
|------------|--------------------------------------------------------------------|-----------|
| <b>II</b>  | <b>Строим релейный компьютер</b>                                   | <b>50</b> |
| <b>13</b>  | <b>Элементы компьютера</b>                                         | <b>51</b> |
| 13.1       | Чем компьютер отличается от калькулятора? . . . . .                | 51        |
| <b>14</b>  | <b>Дешифратор</b>                                                  | <b>52</b> |
| 14.1       | Практикум . . . . .                                                | 54        |
| 14.1.1     | Устройство, угадывающее животных . . . . .                         | 54        |
| 14.1.2     | Ещё больше животных . . . . .                                      | 54        |
| 14.1.3     | Выбор одного из четырёх значений с помощью дешифратора . . . . .   | 54        |
| 14.1.4     | Задачи . . . . .                                                   | 54        |
| <b>15</b>  | <b>Калькулятор с выбором операции</b>                              | <b>55</b> |
| 15.1       | Практикум . . . . .                                                | 56        |
| <b>16</b>  | <b>Регистровый файл</b>                                            | <b>57</b> |
| 16.1       | Практикум . . . . .                                                | 57        |
| <b>17</b>  | <b>АЛУ</b>                                                         | <b>59</b> |
| 17.1       | Практикум . . . . .                                                | 60        |
| <b>18</b>  | <b>Тактовый генератор</b>                                          | <b>63</b> |
| 18.1       | Бесконечная последовательность сигналов . . . . .                  | 63        |
| 18.2       | Копирование значения в цикле из одного регистра в другой . . . . . | 63        |
| 18.3       | Автомат для вычисления чисел Фибоначчи . . . . .                   | 63        |
| <b>19</b>  | <b>ПЗУ</b>                                                         | <b>65</b> |
| <b>20</b>  | <b>Счётчик команд</b>                                              | <b>66</b> |
| 20.1       | Практикум . . . . .                                                | 66        |
| 20.2       | Циклический инкремент регистра . . . . .                           | 66        |
| 20.3       | Циклический инкремент регистра и выборка команды из ПЗУ . . . . .  | 67        |
| 20.4       | Загрузка команды из ПЗУ в регистр . . . . .                        | 67        |
| <b>21</b>  | <b>Набор инструкций</b>                                            | <b>68</b> |
| <b>22</b>  | <b>Декодер инструкций</b>                                          | <b>69</b> |
| <b>23</b>  | <b>Компьютер</b>                                                   | <b>70</b> |
| <b>III</b> | <b>Приложения</b>                                                  | <b>71</b> |
| <b>A</b>   | <b>Как самостоятельно собирать модули</b>                          | <b>72</b> |
| <b>B</b>   | <b>Список деталей для полного набора</b>                           | <b>73</b> |

# Введение

## Как устроена современная электроника

Большинство современных электронных устройств представляют собой компьютеры независимо от того, насколько сложные функции они при этом выполняют. Например, «умная» лампочка загорается, когда становится темно. Лифт едет после нажатия кнопки. А мобильный телефон показывает смешную картинку, если человек нажал на правильное место на экране.

Все эти вещи кажутся очень разными, а на самом деле они очень похожи. Внутри каждого такого устройства есть маленький или большой компьютер. Компьютеры отличаются от других электрических и электронных устройств тем, что работают по программе, которую можно изменить, не переделывая всё устройство целиком.

Программа похожа на кулинарный рецепт: она состоит из последовательности простых шагов. Если взять другую последовательность, то получится другое блюдо, несмотря на одинаковые кастрюли и сковородки.

Сами же простые шаги — это операции с числами: сложение, вычитание, умножение, сравнение и т.п. Например, одним из чисел может быть температура воздуха (которую нужно измерить), а другим — напряжение на обогревателе, который нужно включить, если слишком холодно. Программисты записывают эти вычисления и процесс принятия решений в виде программы, а компьютеры затем программы выполняют.

Все данные, которое есть в компьютерах (телефонах, игровых приставках) представляются в виде чисел. Например, картинка это набор точек, а каждая точка описывается одним или несколькими числами, определяющими её цвет. В свою очередь видео — это множество картинок, дополненных звуковой дорожкой. Звук же можно закодировать последовательностью чисел, описывающих уровень напряжения на воспроизводящем динамике.

Отличаются компьютеры между собой тем, сколько чисел (то есть данных) они могут запоминать и с какой скоростью обрабатывать. Ну и конечно, такими незначительными деталями, как количество пикселей на экране или число кнопок у «мыши».

## Из чего состоит компьютер

Главное, что отличает компьютеры от других механизмов (или электронных устройств вроде часов) — это программы. В компьютере за работу по программе отвечает процессор. Именно он последовательно её «читает» и делает всё, что там написано. Фрагменты программы (шаги, инструкции, команды) обычно очень простые. Например, сложить два числа или перемножить их. Потом уже из этих команд складываются программы, делающие практически что угодно (если программа достаточно сложная).



Кроме процессора в компьютере есть ещё память. Там хранятся все данные пользователя (музыка, фотографии, электронные письма), а также программы для обработки этих данных. Программа обычно хранится в памяти как последовательность команд. Чтобы двигаться по этим командам по мере их выполнения, нужно запоминать номер текущей. За это отвечает счётчик инструкций (program counter, instruction pointer). Счётчик инструкций обычно находится внутри процессора.

Процессор «смотрит» в память на текущую инструкцию и активирует один из своих модулей, отвечающий за её выполнение. Это может быть арифметическая операция, переход к другой части программы (тогда в счётчик инструкций попадает новое значение), передача данных на внешнее устройство (вывод пикселя на экран). После этого счётчик инструкций переключается на следующую команду.

Чтобы проделывать все эти шаги поочерёдно, используется тактовый генератор. Он с заданной частотой посылает сигналы на все остальные модули, в результате чего они активируются один за другим. Например, сначала процессор смотрит в память, потом декодирует инструкцию, то есть, разбирается с тем, какие именно действия нужно будет выполнять дальше, а после выполнения этих действий увеличивает счётчик инструкций.

Все числа (данные), с которыми работает процессор хранятся в памяти. Он может считывать их оттуда и записывать новые. Но внешняя (по отношению к процессору) память обычно работает не слишком быстро, поэтому в процессоре есть несколько регистров, используемых для вычислений. Например, одна команда процессора может сложить числа из двух регистров и записать результат в третий. А потом другая команда отправит значение этого регистра во внешнюю память.

Мы будем строить собственный компьютер с помощью релейного конструктора. То есть формально это будет компьютер, но он будет настолько простой, что можно даже называть его процессором. А в качестве промежуточных этапов у нас будут получаться ещё более простые вычислители.

В целом, компьютер можно сделать из чего угодно: из шестерёнок, реле, транзисторов, микросхем. Электромагнитные реле хороши тем, что сразу понятно, что они делают. Современные компьютеры построены на транзисторах, которые упакованы в микросхемы, но с ними не так интересно играть. Например, реле можно переключить простой отвёрткой, а потом оценить, что изменилось.

Самое главное, что современные процессоры и компьютеры концептуально почти не отличаются от содержимого наших занятий. Мы будем создавать следующие части компьютера (и процессора):

1. Регистры для хранения данных.
2. Вычислительные модули для реализации команд.
3. Память программ.
4. Счётчик инструкций.
5. Логические схемы для декодирования команд и выбора правильных исполнительных модулей.
6. Тактовый генератор.

Такой компьютер сможет выполнять простые программы, например, найти остаток от деления одного числа на другое, используя только сложение и вычитание. А если в какой-то момент реле станет недостаточно, полученные знания можно будет применить для разработки более сложного процессора на современных технологиях или при программировании уже готовых устройств.

**Для чего эта книга и как ей пользоваться**

## Часть I

# Собираем релейный калькулятор

# Глава 1

## Двоичная система счисления

Так уж получилось, что компьютерам и другим цифровым устройствам удобнее всего работать в двоичной системе счисления. Люди привыкли работать с десятичной системой, а компьютеры работают в двоичной. И раз уж мы планируем позаниматься созданием цифровых игрушек, компьютеров и калькуляторов, стоит изучить двоичную систему счисления поподробнее.

Когда мы используем десятичную систему счисления, мы записываем числа с помощью десяти цифр, а у компьютеров числа «записаны» (если так вообще можно говорить) с помощью только двух цифр. Само число остаётся тем же самым независимо от формы записи. Двенадцать стульев будут двенадцатью стульями, если их записать как 12, XII или  $1100_2$ .

Последний вариант это как раз двоичная запись (о чём нам намекает цифра 2 в конце). В ней используется только две цифры: 1 и 0. И этого вполне достаточно, чтобы закодировать любое число. 0 записывается как 0, 1 будет 1 (пока всё легко). Чтобы записать 2 разных цифр уже не хватает, поэтому нужно использовать больше разрядов. Значит 2 — это 10, 3 — 11, 4 — 100, 5 — 101.

Смысл тут такой же, как и в десятичном счёте. Там если разряд равен 9 (последняя доступная цифра), то при увеличении числа он заменяется на 0, а более старший разряд увеличивается на единицу. В двоичном счёте самая большая цифра это 1, поэтому перенос происходит, когда нужно такой разряд увеличить. Тогда при сложении 11 и 1 получается 100: сначала переполняется младший разряд, а затем из-за переноса и следующий за ним.

### 1.1 Перевод числа из двоичной системы счисления

Дальше мы будем часто встречать двоичные числа (например, наш калькулятор будет показывать именно двоичные числа). Но как их понимать? Как из двоичной записи получить десятичную? Можно просто запомнить все нужные значения. Например,  $1111_2 = 15_{10}$ . Но вдруг понадобятся числа больше чем из четырёх разрядов? Неплохо бы понять принцип.

Когда к числу добавляется ноль справа, это то же самое, что умножить исходное число на 10. Очевидно это так и для десятичной, и для двоичной систем счисления. Только в десятичной 10 это 10, а в двоичной 10 это 2.

Получается, что последовательным домножением единицы на  $10_2$  можно получать степени двойки:

$$1_2 = 2^0 = 1$$

$$10_2 = 2^1 = 2$$

$$100_2 = 2^2 = 4$$

$$1000_2 = 2^3 = 8$$

$$10000_2 = 2^4 = 16$$

А любое двоичное число можно разобрать на полученные выше слагаемые:

$$1101_2 = 1000_2 + 100_2 + 1_2 = 2^3 + 2^2 + 1 = 8 + 4 + 1 = 13$$

Один разряд двоичного числа называется битом. С помощью одного бита можно закодировать два варианта чего либо («да» или «нет», «чёрное» или «белое» и т.п.), так как такой разряд может содержать либо 0, либо 1.

Если к одному биту добавить ещё один, число возможных комбинаций увеличится вдвое. Так можно записать информацию о четырёх событиях или состояниях (00, 01, 10, 11). Если взять три бита, то уже о восьми и так далее. Дальше мы будем в основном работать с четырёхбитными числами, то есть от 0000 до 1111 (или от 0 до 15).

## 1.2 Перевод числа в двоичную систему счисления

Если нам нужно в уме перевести десятичное число (например, 18) в двоичную систему счисления, то можно действовать так. В начале алгоритма число-результат у нас равняется нулю. Находим наибольшую степень двойки, которая «помещается» в десятичное число (это будет  $16 = 2^4 = 10000_2$ ). Потом вычитаем это число из исходного, а к результату добавляем единицу в нужном разряде (получится  $18 - 16 = 2$  и  $0 + 10000_2 = 10000_2$ ). Повторяем эти два шага снова (результатом будет  $2 - 2 = 0$  и  $10000_2 + 10_2 = 10010_2$ ). Исходное число обнулилось, ничего дальше делать не нужно.

Почему этот способ сработает? Не может ли дважды получиться единица в одном и том же месте? Оказывается нет, потому что степень двойки умноженная на два (если два раза вычитать одно и то же), это будет следующая степень двойки. А мы ведь вычитали наибольшую из возможных степеней, поэтому такого быть не может.

Этот способ неплох, но на практике часто делают наоборот — находят разряды результата начиная с младшего из них. Как найти самый младший разряд? Если посмотреть на двоичную запись, то становится очевидным, что этот разряд будет равен остатку от деления на  $10_2$ . Чтобы этот разряд теперь отбросить и перейти к следующему, всё число нужно разделить на  $10_2$ .

Делим 18 на  $10_2 = 2$ : младшая цифра (остаток) будет 0, частное равно 9. Прделаем это несколько раз:

$$\begin{array}{rcl} 18 : 2 & = & 9 \quad (\text{остаток } 0) \\ 9 : 2 & = & 4 \quad (\text{остаток } 1) \\ 4 : 2 & = & 2 \quad (\text{остаток } 0) \\ 2 : 2 & = & 1 \quad (\text{остаток } 0) \\ 1 : 2 & = & 0 \quad (\text{остаток } 1) \end{array}$$

Теперь выписываем полученные остатки, начиная с последнего, и получаем двоичную запись числа 18: 10010.

### 1.3 Двоичная логика

Компьютеры должны не только хранить числа, но и ещё что-то делать с ними. Люди обычно пользуются сложением, вычитанием, умножением и делением. Для процессора эти действия слишком сложные, поэтому они выполняются на основе более простых операций.

Операции выполняются над двоичными числами. То есть у каждой есть один или несколько операндов-двоичных чисел, а результатом обычно будет одно число (ну и может иногда ещё пара дополнительных битов).

Элементарные операции, из которых можно построить более сложные, вообще оперируют единичными битами. Например, два бита-операнда на входе и один бит-результат на выходе.

Чтобы описать, как работает операция, обычно приводят так называемую таблицу истинности. В ней для каждой комбинации значений операндов ( $A$  и  $B$ ) записан результат операции ( $R$ ). Примерно так:

| A | B | R |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 1 |

Например, вторая строка таблицы означает, что если  $A = 1$  и  $B = 0$ , то результатом  $R$  будет 1.

Вариантов таких операций (и соответственно таблиц) всего 16 (по числу комбинаций единиц и нулей на выходе). В реальности используются не все из них. Некоторые (самые полезные) мы сейчас рассмотрим.

Самая простая операция (или оператор) — это отрицание. У неё только один операнд и она меняет его значение на обратное. Записывать эту операцию мы будем как *not*  $A$ ,  $\neg A$  или  $\sim A$ .

| A | $\sim A$ |
|---|----------|
| 0 | 1        |
| 1 | 0        |

Логическое «или» (дизъюнкция, логическое сложение) выдаёт в качестве результата единицу, если хотя бы один из операндов равен единице (немного отличается от обычного сложения, потому что из 1 и 1 не получается 2). Обозначается такой оператор как *A or B*,  $A \vee B$  или  $A \mid B$ .

| A | B | $A \mid B$ |
|---|---|------------|
| 0 | 0 | 0          |
| 1 | 0 | 1          |
| 0 | 1 | 1          |
| 1 | 1 | 1          |

Логическое «и» (конъюнкция, логическое умножение) выдаёт единицу только если на обоих входах будет единица (и правда похоже на умножение). Обозначаться такой оператор может как *A and B*,  $A \wedge B$  или  $A \& B$ .

| A | B | A & B |
|---|---|-------|
| 0 | 0 | 0     |
| 1 | 0 | 0     |
| 0 | 1 | 0     |
| 1 | 1 | 1     |

Для создания компьютера перечисленных операций уже достаточно. Но если процессор будет делать ещё некоторые другие, то будет удобнее писать программы (или создавать аппаратные устройства, используя те же схемы).

Одной из таких дополнительных операций будет «исключающее или» (сложение по модулю 2, оператор неравенства). Она даёт нам единицу только для случаев, когда один операнд не совпадает с другим (то есть в одном единица, а в другом ноль)<sup>1</sup>. Второе название, сложение по модулю 2, означает, что результатом будет последняя цифра суммы операндов (в двоичном виде, естественно). Обозначается эта операция как  $A \text{ xor } B$ ,  $A \oplus B$  или  $A \wedge B$ .

| A | B | A $\wedge$ B |
|---|---|--------------|
| 0 | 0 | 0            |
| 1 | 0 | 1            |
| 0 | 1 | 1            |
| 1 | 1 | 0            |

С помощью логических операций компьютеры могут вычислять принимать решения вроде «если число равно нулю» И «выполняется команда деления», то зажечь лампочку «ошибка». Конечно, это слишком элементарные операции (да ещё и с однокбитовыми числами). Как делать что-то сложнее, например сложение или вычитание, мы узнаем позднее.

Но нельзя не упомянуть ещё одну важную операцию, на которой строятся вычисления в современных компьютерах. Правда, в релейном конструкторе её нет, но эта технология не слишком-то современная.

Это операция «и-не», то есть логическое умножение в паре с отрицанием (ещё она называется штрихом Шеффера):

| A | B | $\sim(A \& B)$ |
|---|---|----------------|
| 0 | 0 | 1              |
| 1 | 0 | 1              |
| 0 | 1 | 1              |
| 1 | 1 | 0              |

Оказывается, с помощью одной этой операции можно выразить любую другую. Обозначим здесь «и-не» с помощью кружочка  $\circ$ :

- $\sim A = A \circ A$
- $A \& B = \sim(A \circ B)$
- $A | B = \sim A \circ \sim B$

<sup>1</sup>С помощью «исключающего или» компьютеры делают простейший контроль корректности передачи данных. Вместо, к примеру, восьми бит числа, которое нужно передать, в канал связи отправляется девять. Девятый бит (бит чётности) выбирается таким образом, чтобы в полученном девятибитном числе было чётное число единиц. И если принимающая сторона увидит, что число таких бит нечётно, значит произошла ошибка. А вычислить значение бита чётности можно применив «исключающее или» ко всем значащим битам передаваемого числа.

## 1.4 Задачи

1. Напишите программу для перевода числа из десятичной системы в двоичную на любом известном вам языке программирования.
2. Придумайте как вычислять «исключающее или» используя только «и-не».
3. Найдите другую операцию (кроме «и-не»), которую можно использовать несколько раз в одном выражении, чтобы получить все остальные операции («и», «или», «не», «исключающее или»).



## Глава 2

# Релейный конструктор

### 2.1 Выключатели и двоичная логика

Первые компьютеры были построены на электромагнитных реле. А реле — это управляемый электрическим сигналом контакт. То есть это такие переключатели (вроде выключателей света, которыми мы пользуемся дома), но управляемые не нажатием вручную, а с помощью электрического сигнала.

Выключатели и электрический ток в проводах очень хорошо подходят для работы с двоичным кодом. Кнопка нажата — значит 1, а не нажата — 0. Ток течёт — 1, не течёт — 0.

Например, если на стене расположены 4 выключателя, то с их помощью можно закодировать четырёхбитное число. Переключив их в нужные положения мы «запишем» четырёхбитное число. А посмотрев на лампочки, соединённые с выключателями (условимся, что на выключатели смотреть нельзя), мы можем узнать, что было записано. А если на лампочки посмотрит наш друг с другого конца комнаты, то это будет не просто считывание данных, но ещё и передача их на другое «устройство».

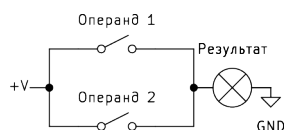
Получается, что запоминать и считывать данные мы можем. А как проводить вычисления? Хотя бы самые простые логические операции.

Например, мы хотим выполнять логическое «и» ( $A \text{ and } B$ ): если два выключателя нажаты, то лампочка загорается, а если хотя бы один разомкнут, лампочка гаснет. Значит операндами являются состояния выключателей (нажаты или нет), а результатом состояние лампочки (горит или нет). Этого можно достичь с помощью последовательного соединения:



Слева на схеме источник напряжения, а справа «земля», которая где-то за пределами рисунка соединена с минусовым контактом источника напряжения. Если ток будет течь слева направо через всю цепь, от плюса к минусу, он заставит светиться лампочку «Результат». Но ток потечёт, если только оба выключателя «Операнд 1» и «Операнд 2» будут замкнуты. Если незамкнутым выключателям сопоставить 0, а замкнутым 1, то мы получим, что эта схема выполняет функции логического «и».

Логическое «или» ( $A \text{ or } B$ ) тоже сделать легко. Ведь ток должен течь в цепи если любой из выключателей-операндов нажат. Тогда мы сможем достичь такого результата соединив их параллельно:



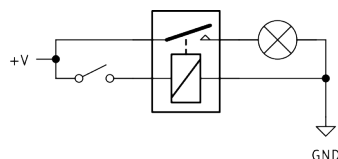
А что будет, если мы захотим объединить несколько операций в одной схеме? Например, выход со схемы «или» подать на вход схемы «и» (то есть, хотим посчитать что-то вроде  $(A \text{ or } B) \text{ and } C$ ). Так как выход у нас это лампочка, а вход — выключатель, то придётся поставить туда человека, который будет смотреть на лампочку и нажимать на нужный выключатель в следующей схеме.

Вот если бы лампочка сама могла «нажимать» на выключатель. . . И это действительно возможно, только вместо лампочки нужно взять электромагнит. Так мы сделаем электромагнитное реле.

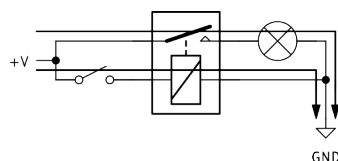
## 2.2 Электромагнитное реле

Электромагнитное реле — это электромагнит, один или несколько переключателей и якорь (подвижная часть для передачи усилия на переключатели). При прохождении тока через обмотку электромагнита он притягивает якорь и нажимает на выключатели. С помощью этого механизма можно передавать сигналы с выхода одной схемы на вход другой.

На рисунке ниже схема, в которой есть одно реле (большой прямоугольник). Внутри этого реле находится электромагнит (перечёркнутый прямоугольник) и переключаемый контакт. К электромагниту подключена кнопка, позволяющая подавать на него напряжение, а к переключаемому контакту реле подключена лампочка.



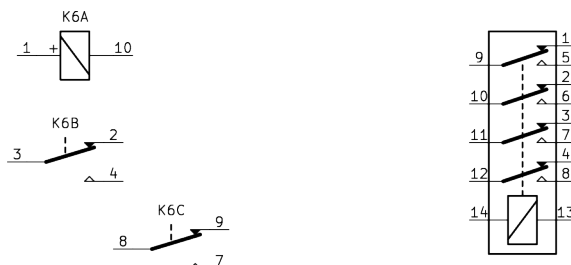
Когда кнопка не нажата, ничего не происходит. Ток не течёт ни через обмотку электромагнита, ни через лампочку. Если же кнопку нажать, на электромагните окажется напряжение и он притянет якорь и подвижную часть контакта, замкнув его. Таким образом цепь с лампочкой тоже замкнётся и лампочка загорится.



Когда логические операции реализуются при помощи электромагнитных реле, в качестве логической единицы обычно используется наличие напряжения и протекание тока (цепь замкнута), а для логического нуля — отсутствие напряжения (цепь разомкнута).

В конструкторе применяются электромагнитные реле с четырьмя переключающими контактами. Чаще всего в схемах электромагнит и переключающие контакты изображаются отдельно (как на следующем рисунке слева). Это может быть удобно, если управляющая цепи и управляемая не связаны между собой электрически (для этого обычно и нужны реле).

Мы же будем использовать более наглядное обозначение (оно приведено справа на следующем рисунке), когда и электромагнит и контакты объединяются одним прямоугольником. Это оправданно ещё и тем, что в наших схемах контакты и катушка зачастую включены в одну цепь.

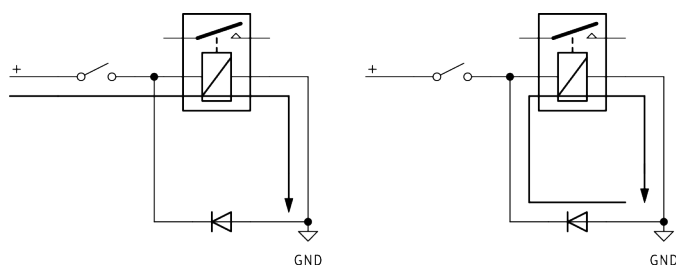


Контакты 13 и 14 подключены к катушке, управляющей состоянием контактов. Когда напряжение на катушку не подано (и ток через неё не течёт), контакты переключателей соединены так: 9 – 1, 10 – 2, 11 – 3, 12 – 4.

Если же реле включится, подвижная часть контактов сдвинется, разомкнув цепи, описанные выше, и замкнув следующие: 9 – 5, 10 – 6, 11 – 7, 12 – 8.

В каждое реле конструктора встроен светодиод, поэтому можно легко увидеть, когда оно включается. Это помогает отлаживать схемы или наблюдать хранящиеся в релейной памяти значения.

Во всех модулях параллельно каждой из катушек реле подключён диод в обратном направлении. Это сделано для предотвращения искрения контактов других реле (в цепи которых подключена катушка) при их размыкании. Искрение возникает из-за того, что несмотря на разрыв цепи из-за индуктивности катушки ток должен продолжать течь какое-то время. Но из-за разрыва это невозможно в обычном режиме, поэтому возникает электрическая дуга, постепенно разрушающая контакты.



На рисунке подверженный искрению контакт — это переключатель слева. Он может быть сам по себе, а может и принадлежать какому-то другому реле. Чтобы контакты этого переключателя меньше портились и нужен диод. Оставшийся электрический импульс проходит уже через него, а не через разомкнутые контакты.

## 2.3 Модули

Релейный конструктор — это не просто реле и провода, как можно было бы подумать, а несколько видов готовых модулей, выполняющих простые функции. Каждый модуль состоит из печатной платы, нескольких разъёмов на ней и других компонентов, в зависимости от его функций (реле, переключатели, диоды, резисторы).

С готовыми модулями работать намного удобнее, чем с отдельными диодами, реле и резисторами, которые ещё нужно чем-то соединить. Модули похожи на простые микросхемы: один складывает числа, другой хранит несколько битов информации, и так далее. Ещё есть модуль с переключателями, чтобы человек мог управлять схемой.

В следующих главах мы будем постепенно знакомиться с каждым из модулей, соединять их друг с другом, чтобы делать вычислительные устройства: несложные логические схемы, игрушки, калькулятор и в конце концов компьютер.

## 2.4 Соединительные разъёмы

Модули соединяются между собой с помощью разъёмов на краях плат, как детали конструктора. Обычно неподалёку от каждого их таких разъёмов находятся ещё и парные им разъёмы для плоских кабелей. Ими можно воспользоваться, чтобы соединить два модуля, которые не получается состыковать иначе.

В каждом разъёме для подключения платы или кабеля есть контакты для питающего напряжения (контакт 1) и нуля (контакт 8). Поэтому все разъёмы электрически совместимы друг с другом, но логически их соединение не всегда имеет смысл.

Если какой-то разъём не используется для передачи данных, к нему можно подключить источник питания. А остальные модули запитаются за счёт того, что контакты питающего напряжения и нуля есть во всех разъёмах.

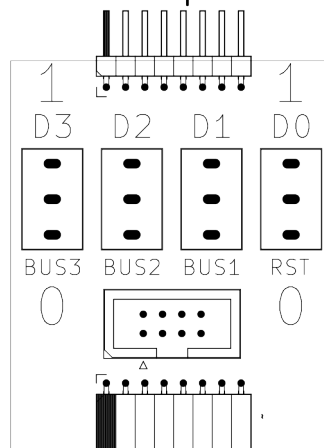
Кроме питающего напряжения через соединения передаются и полезные сигналы, занимающие до четырёх контактов (номера от 2 до 5), в зависимости от назначения модуля. Они используются для кодирования четырёхбитного числа или передачи четырёх управляющих сигналов разного назначения.

Контакты 6 и 7 в текущей версии конструктора не используются.

## 2.5 Модуль переключателей

Модуль с переключателями (или тумблерами) используется для установки уровней 0 или 1 на выходном разъёме вручную. Присоединяя его к разъёмам других модулей, можно подавать им на вход четырёхбитное число, либо переключать до четырёх управляющих сигналов.

### Положение переключателей



### Положение переключателей

Входных разъёмов у модуля с тумблерами нет, ведь никакие данные в него не поступают, чтобы переключить реле или зажечь светодиод. Ведь на плате ничего этого нет. Поэтому у модуля есть только выходные разъёмы, на которые попадают сигналы с тумблеров. Одноимённые контакты этих разъёмов соединены между собой. Можно синхронно управлять одним набором переключателей сразу несколькими модулями, если задействовать нужное количество выходных разъёмов модуля переключателей.

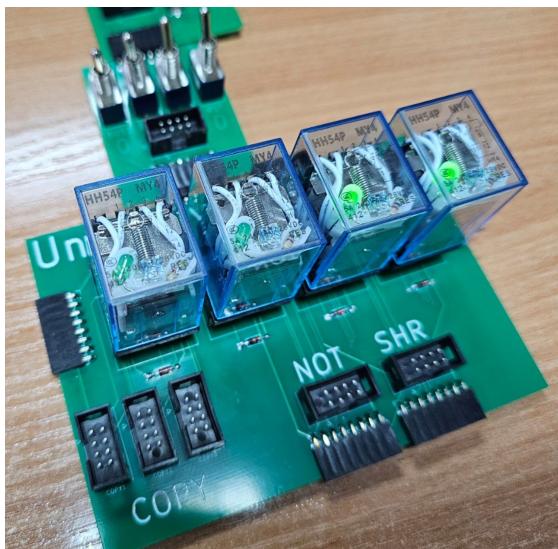
## 2.6 Практикум

Проще всего проверить работу тумблеров, подключив их к модулю унарных логических операций.

Список модулей:

- Модуль переключателей: 1 штука
- Модуль унарных логических операций: 1 штука
- Проводной шлейф для соединения модулей: 1 штука

Сначала соберём следующую схему.



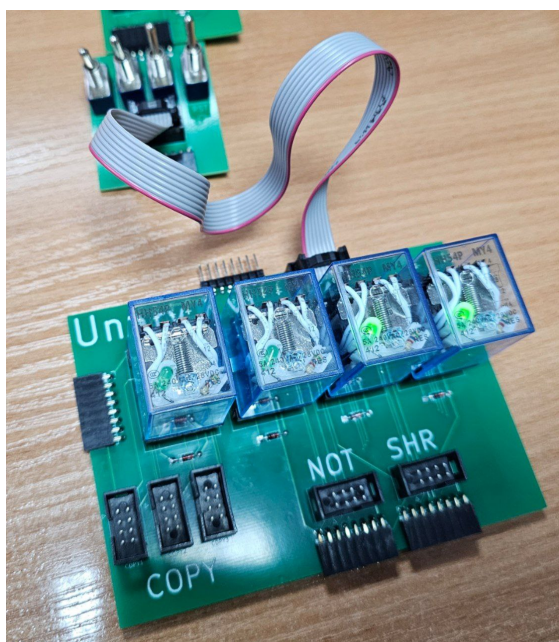
Источник напряжения можно подключить к любому незадействованному разъёму, потому что контакты для питания схемы есть на всех разъёмах.

Обычно подключать источник питания нужно последним, но если нужно переставить какой-то модуль в другое место или добавить какой-то проводник, то можно это делать и при подключённом питании. Ничего не сломается.

1. Попереключайте тумблеры. Убедитесь, что положение одного тумблера меняет состояние одного реле.
2. Запомните включенное и выключенное состояния тумблеров, а также их порядок по отношению к реле, чтобы позднее не было проблем с управлением другими схемами.

Как видите, реле переключаются, в зависимости от положений тумблеров. При этом реле ещё и генерируют выходные сигналы, но с ними мы поработаем позднее.

Теперь заменим прямое соединение двух плат на соединение с помощью плоского кабеля.



Поперекл­ю­чай­те тумб­ле­ры как рань­ше, что­бы убе­д­ить­ся, что всё ра­бо­та­ет рав­но так же. По­это­му даль­ше вы­би­рай­те тот спо­соб под­клю­че­ния, ко­то­рый бу­дет вам удо­б­нее — они рав­но­знач­ны.

## 2.7 Задачи

1. При­ду­май­те, как со­еди­нить два мо­ду­ля с тумб­ле­ра­ми для вы­пол­не­ния ло­гичес­ко­го ИЛИ.

## Глава 3

# Хранение и передача данных

### 3.1 Ячейка памяти

Сохранять и считывать данные несложно. Например, можно записать единички и нолики на бумаге и прочитать, когда потребуется вспомнить эту последовательность.

Но для автоматического устройства, которое мы хотим создать, бумага и ручка подходят плохо. Можно «записать» единички и нолики, набрав их на тумблерах. «Считать» сохранённую в их положениях информацию можно подключив тумблеры к электрической цепи: например, зажечь лампочки или даже отправить запомненные символы электронным письмом.

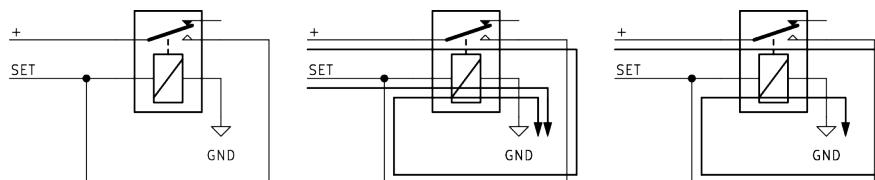
Но кто будет переключать все эти тумблеры? Хотелось бы, чтобы при появлении сигнала во входной цепи, его можно было запомнить без участия человека (операция записи), а затем, через произвольное время, когда входной сигнал уже пропал, получить такой же сигнал в выходной цепи (операция чтения).

То, что нам нужно, называется ячейкой памяти. Ячейки могут отличаться объёмом хранимой информации, способами управления и элементной базой. Сейчас мы рассмотрим такую ячейку, построенную на электромагнитных реле.

### 3.2 RS-триггер

Хранение одного бита можно сделать с помощью одного реле. Если оно включается с помощью внешнего сигнала, то реле «сохраняет» единицу. Но когда внешний сигнал пропадает, сохранённая единица пропадает тоже, потому что на катушке реле больше нет напряжения.

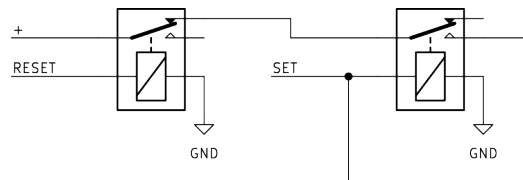
Чтобы включённое состояние запоминалось, нужно организовать обратную связь через один из контактов реле. Как только реле включается с помощью входа *SET*, на его обмотку подаётся напряжение через его же замкнутый контакт. Поэтому даже если перестать подавать напряжение на вход *SET*, реле всё равно останется включённым:





Но если такое реле уже включилось, выключить его не получится. Единственный способ — это отключить питание всей схемы. Тогда контакты разомкнутся и при повторной подаче питания реле будет в выключенном состоянии.

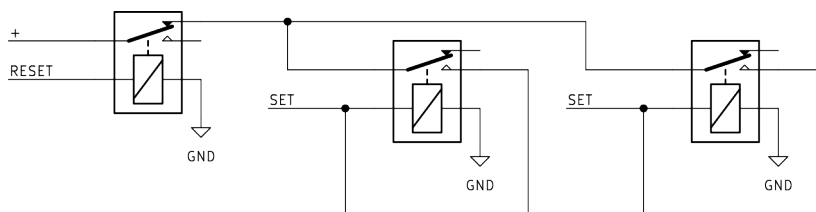
Чтобы не отключать всю схему, нужно предусмотреть отдельное реле, позволяющее отключить питание только для запоминающего реле:



Теперь подав сигнал на вход *RESET*, можно выключить реле, которое «запомнило» единицу. И на его выходе опять окажется ноль.

Устройство (ячейка памяти) для хранения одного бита информации называется триггером. Чтобы его сделать нам потребовалось два реле, но когда мы говорим о состоянии триггера, то речь идёт о том, включено или выключено то самое «запоминающее» реле. Например, если оно включено, то в триггере хранится единица.

Триггер с двумя входами *RESET* и *SET* называется RS-триггером. На практике удобно объединить несколько таких триггеров в многобитовый регистр и использовать для них единый сигнал сброса, сэкономив несколько реле:



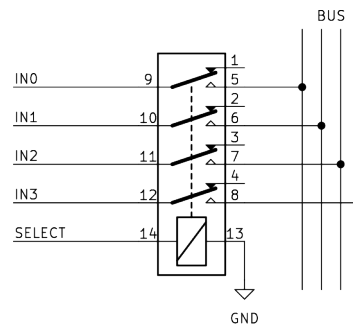
### 3.3 Шина

Шина — это набор проводников, объединённых общим назначением. К этим проводникам подключаются несколько устройств для обмена сигналами.

Например, к внутренним шинам компьютера могут быть подключены память и жёсткий диск. Тогда по шине данных передаются данные между процессором и памятью или процессором и жёстким диском, шина адреса используется для выбора определённой ячейки в памяти, а по шине управления приходят сигналы, позволяющие отличать обращение к памяти от обращения к диску.

Так как к шине обычно подключается больше двух устройств, необходимо изолировать те из них, которые не участвуют в обмене данными в конкретный момент времени. Например, когда процессор взаимодействует с памятью через шины данных или адреса, жёсткий диск не должен подключаться к тем же шинам. В противном случае сигналы получатся некорректными, а может одно из устройств даже сгорит.

В релейном компьютере мы можем использовать для подключения к четырёхбитной шине шинный формирователь из одного реле:



Когда реле включается, оно соединяет подключённое к нему устройство с шиной. Таким устройством может быть набор из нескольких триггеров (позднее мы будем называть его регистром). Подключение будет двунаправленным, поэтому с помощью этой схемы можно как записывать данные в регистр, так и считывать оттуда.

Таким образом, сигнал *SELECT* подаётся на шинные формирователи только тех устройств, которые должны повзаимодействовать через шину. Остальные устройства, подключённые к шине, остаются изолированными от неё.

### 3.4 Регистр

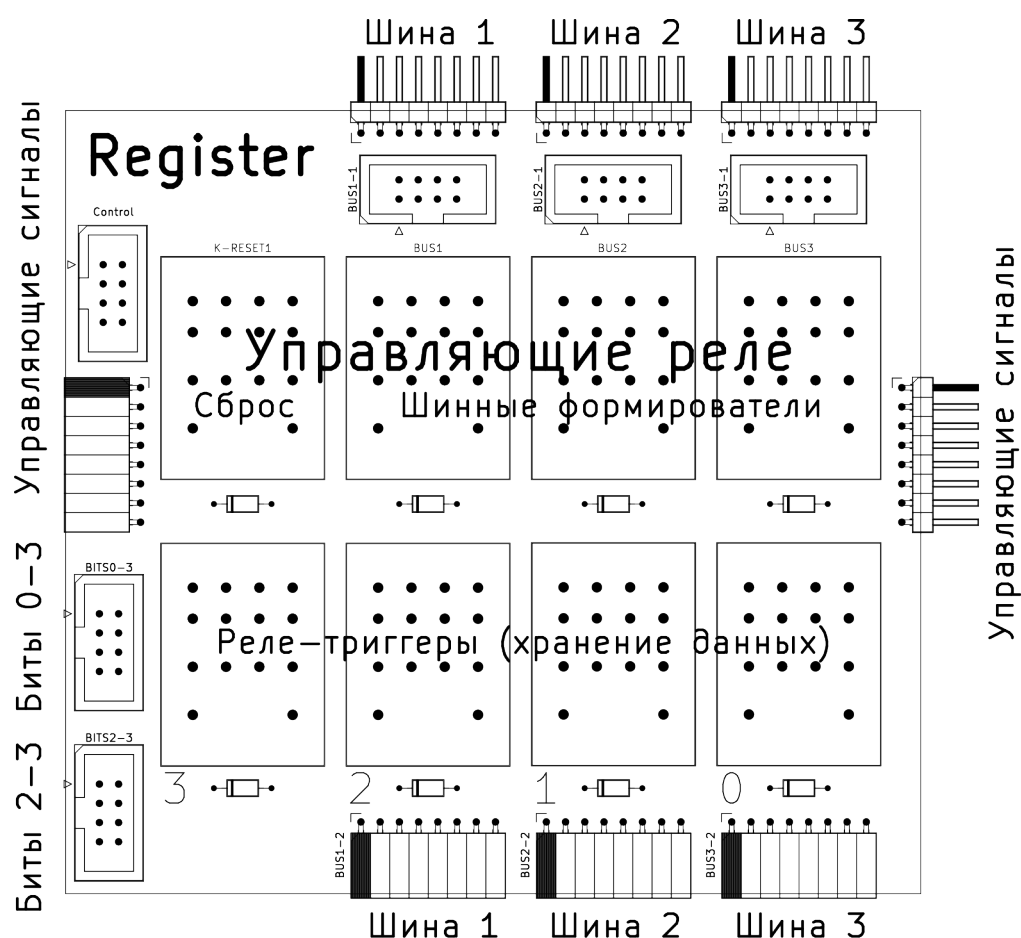
Регистры внутри процессоров (или периферийных устройств) используются для хранения данных и для проведения вычислений. Регистры могут иметь выделенное назначение (например, хранить режим работы устройства или счётчик инструкций), а могут быть универсальными (в разные моменты хранить адреса, операнды арифметических операций, значения для ввода-вывода).

### 3.5 Регистровый модуль

Для хранения данных в регистровом модуле используются RS-триггеры, описанные выше, но сигнал для сброса у них общий. Таким образом, записывать данные можно только во все четыре бита сразу. Поэтому такой модуль и реализует функции цельного регистра, а не нескольких разрозненных RS-триггеров.

Также модуль регистра содержит три шинных формирователя. Их можно использовать для подключения триггеров к разным устройствам через три шины. Например, одна шина может быть предназначена для первого операнда при вычислениях, вторая для второго операнда, а третья для копирования данных между регистрами.

Выглядит плата регистрового модуля так:



Если на плату регистра установить только реле шинных формирователей, получится модуль, который может подключать четырёхбитные сигналы (полученные из разъёма с обозначением 0 – 3 для прямого чтения значений триггеров) к одной из выбранных шин.

На плате регистра есть следующие разъёмы:

- Слева и справа: управляющие сигналы сброса и выборки. Можно подключить тумблеры для ручного включения сигналов. Также можно соединить несколько модулей регистра, чтобы управлять одним набором сигналов сразу для 8, 12 ... бит.
- Сверху и снизу: три шины данных. Реле регистра могут подключаться к шинам для записи или чтения данных.
- Разъёмы с битами 0 – 3 и 2 – 3 для прямого чтения или записи триггеров без подключения к шине через шинный формирователь.

Чтобы прочесть значение регистра нужно активировать сигнал выборки его на нужную шину. А уже через шину сигналы поступят на вход какой-то другой схемы.

Для записи значения в регистр нужно сначала обнулить его триггеры, иначе запись не произойдёт в тех битах, где уже хранятся единицы. После этого регистр подключается к нужной шине, и триггеры защёлкивают нужное значение.

### 3.5.1 Практикум

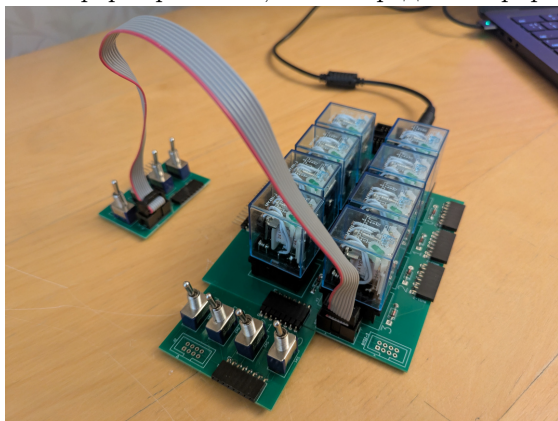
#### Работа регистрового модуля

Список модулей:

- Модуль переключателей: 2 штуки
- Регистровый модуль: 1 штука

#### Регистр без шины

Сначала соберём схему, в которой не используется соединение регистра и тумблеров через шинный формирователь, чтобы продемонстрировать, что шина всё-таки нужна.

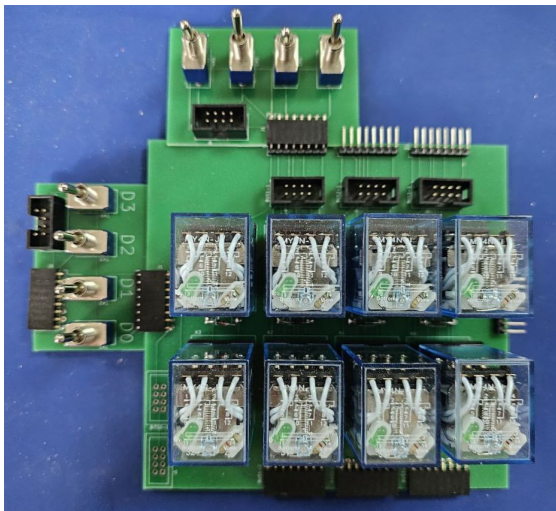


1. Подключите тумблеры кабелем (для этого разъёма нет другого варианта) к битам 0 – 3 вместо шины. Это позволит нам управлять включением триггерных реле непосредственно с помощью тумблеров.
2. Набирайте значение на тумблерах, подключенных кабелем. Убедитесь, что биты переключаются в 1, но не возвращаются в 0, если тумблер переключить обратно.
3. Включите и выключите сигнал сброса (тумблеры, подключенные к боковому разъёму). Убедитесь, что для тех тумблеров, что переключены в 1, соответствующие реле не сбрасываются.
4. Обнулите тумблеры, управляющие триггерами (те, что подключены проводом).
5. Включите и выключите сигнал сброса. Убедитесь, что значения всех битов теперь 0.

#### Регистр с шиной

Неудобство первого варианта схемы в том, что приходится выключать все тумблеры, чтобы можно было просто сбросить все биты регистра. А ведь это могут быть не тумблеры, а какое-то другое устройство (к примеру, ещё один регистр), который может быть не согласен сбрасываться.

Поэтому изменим схему следующим образом:



- Тумблеры слева управляют работой регистра. Бит 0 — сброс триггеров регистра, бит 1 — выборка на шину 1. Тумблеры 2 и 3 здесь не используются.
- Тумблеры сверху нужны для ввода значения регистра. Когда он подключается к шине 1, значение, набранное на тумблерах, записывается в регистр.

Теперь можно проверить, что изменилось в работе схемы:

1. Отключите все управляющие сигналы.
2. Наберите значение на тумблерах для данных. Убедитесь, что это не влияет на регистр.
3. Включите и выключите сигнал выборки на шину 1. Убедитесь, что данные записались в регистр.
4. Включите и выключите сигнал сброса. Убедитесь, что значения всех битов теперь 0. Состояния тумблеров, присоединённых к шине, никак не помешали выполнить сброс.

### 3.5.2 Задачи

1. Возьмите два регистровых модуля и придумайте, как скопировать данные из одного регистра в другой, не запоминая их в голове.

## Глава 4

# Хранение больше четырёх бит

### 4.1 Регистровый файл

Несколько регистров можно соединить в регистровый файл. Регистровый файл — это набор регистров, которые принадлежат одному устройству. Например, в регистровом файле процессора будет счётчик инструкций, регистры для хранения арифметических операндов, а также для хранения адреса при обращении к памяти.

Регистры объединены в регистровый файл, потому что процессор при выполнении операций делает с регистрами похожие действия. Копирует один в другой, берёт их в разном порядке для вычислений, записывает в память. Поэтому все регистры должны находиться «рядом», чтобы схема работы с ними не была слишком сложной.

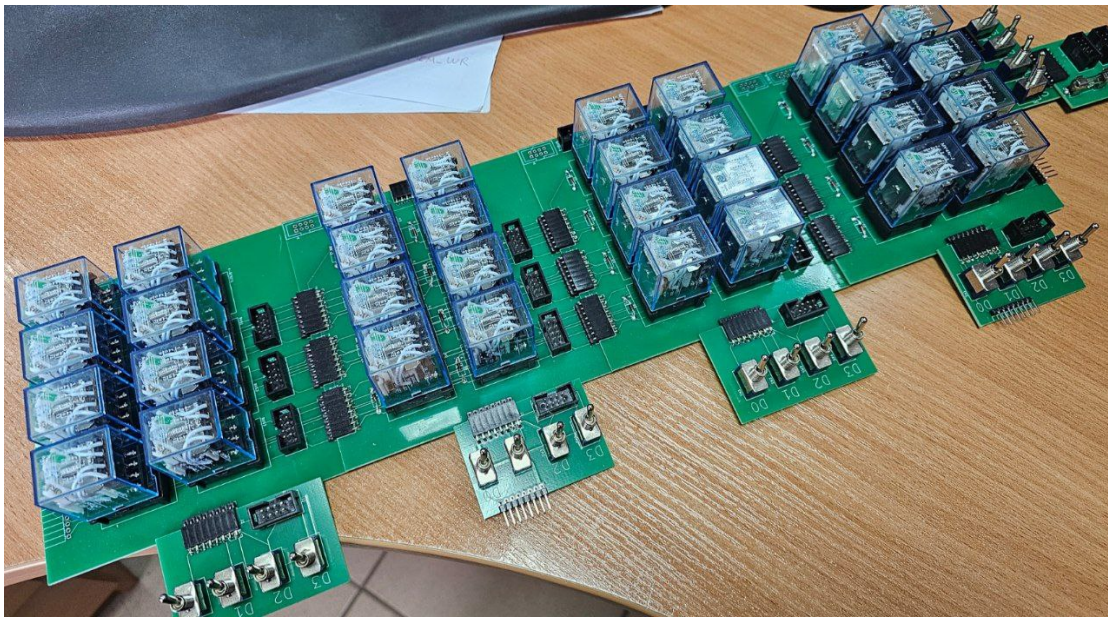
В нашем конструкторе у каждого из регистров есть сигналы выборки на одну из трёх шин. Для одного регистра эти сигналы могут активироваться как поочерёдно, так и одновременно (хотя второе требуется редко). Если два регистра подключены к одной шине одновременно, то значения одного копируются в другой. Если точнее, включённые биты включают аналогичные в другом регистре, то есть копирование происходит в обе стороны одновременно.

Нулевые биты при этом копироваться не могут. Для записи нулей регистр необходимо сбросить.

#### 4.1.1 Практикум

Список модулей:

- Модуль переключателей: 5 штук
- Регистровый модуль: 4 штуки



Запись в регистры:

1. Отключите все управляющие сигналы.
2. Наберите значение на тумблерах, подключённых к шине данных 1.
3. Подключите с помощью тумблера регистр к шине 1. Убедитесь, что в него записалось набранное значение.
4. Отключите регистр от шины.
5. Установите другое положение тумблеров, подключённых к шине данных.
6. Подключите любой другой регистр к шине 1. Убедитесь, что в него записалось набранное значение.

Копирование значения:

1. Отключите все управляющие сигналы.
2. Подключите регистр с ненулевыми битами к шине 2.
3. Подключите пустой регистр к шине 2. Убедитесь, что он получил такое же значение, что и в первом регистре.
4. Отключите все управляющие сигналы.
5. Аналогично проверьте шину 3.

#### 4.1.2 Задачи

1. Пронумеруйте в уме регистры слева направо или сверху вниз (смотря как у вас лежит плата на столе). Теперь сделайте так, чтобы в регистр 1 попало значение 0001, в регистр 2 значение 0010, в регистр 3 значение 0011 и так далее.

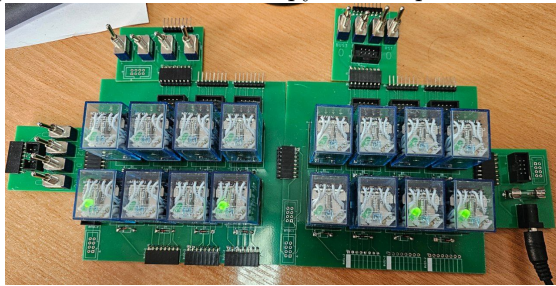
## 4.2 Восьмибитные регистры

У регистров и вычислительных модулей справа и слева есть разъёмы для расширения разрядности.

Для регистров через эти разъёмы передаются сигналы сброса и подключения к шинам. Поэтому один набор тумблеров может использоваться для двух четырёхбитных плат-регистров, если они соединены в один восьмибитный регистр.

### 4.2.1 Практикум

Соедините два регистра и подключите к их шинам 1 по модулю с переключателями. Ещё один модуль с переключателями нужно присоединить сбоку к одному из регистров для управления сигналами загрузки и сброса.



1. Отключите все управляющие сигналы.
2. Наберите входные данные 00111001.
3. Подключите тумблеры с данными к шине 1, подав соответствующий управляющий сигнал.
4. Отключите регистры от шины.
5. Убедитесь, что восьмибитное значение сохранилось
6. Подайте сигнал сброса на регистры.
7. Убедитесь, что оба регистра обнулились.



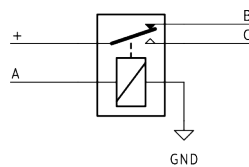
## Глава 5

# Унарные логические операции

### 5.1 Отрицание

Одна из самых простых операций, которая используется в современных компьютерах — это операция отрицания. Она преобразует единицу в ноль, а ноль в единицу.

Такую логику можно реализовать с помощью нормально замкнутого контакта реле:



Когда реле выключено, ток может течь через контакт, поэтому на выходе *B* получается единица. Если же на входе *A* появляется единица, реле переключается и ток через нормально замкнутый контакт не течёт. На выходе *B* теперь ноль.

### 5.2 Копирование

На схеме выше кроме выхода *B* с инвертированным значением есть ещё выход *C*. Значение, соответствующее этому сигналу, совпадает со значением на входе схемы (*A*).

На первый взгляд это кажется ненужным, ведь можно просто использовать *A* везде, где потребуется. Почти всегда так и есть, но как мы позже увидим, иногда нужно соединить вместе несколько выходных сигналов. И вот тогда, чтобы они не влияли друг на друга (ведь цепь получается одна), используется такое «копирование», подобное тому, что на этой схеме.

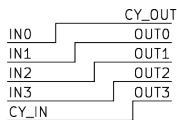
### 5.3 Сдвиг

Операция сдвига ещё более простая по сравнению с отрицанием. Суть её в том, что биты в числе перемещаются по какому-либо принципу (влево, вправо, по кругу).

В конструкторе реализован сдвиг вправо. Нулевой бит вытесняется из числа (и может использоваться отдельно). Первый бит становится нулевым, второй первым и так далее.

Старший бит становится вторым по значимости, а на его место приходит ноль, либо вытесненный нулевой бит из соседнего модуля. Такой сдвиг эквивалентен делению на два, поэтому деление на степень двойки в программах процессор вычисляет с помощью инструкции сдвига.

Для реализации сдвига вправо можно даже не использовать реле:

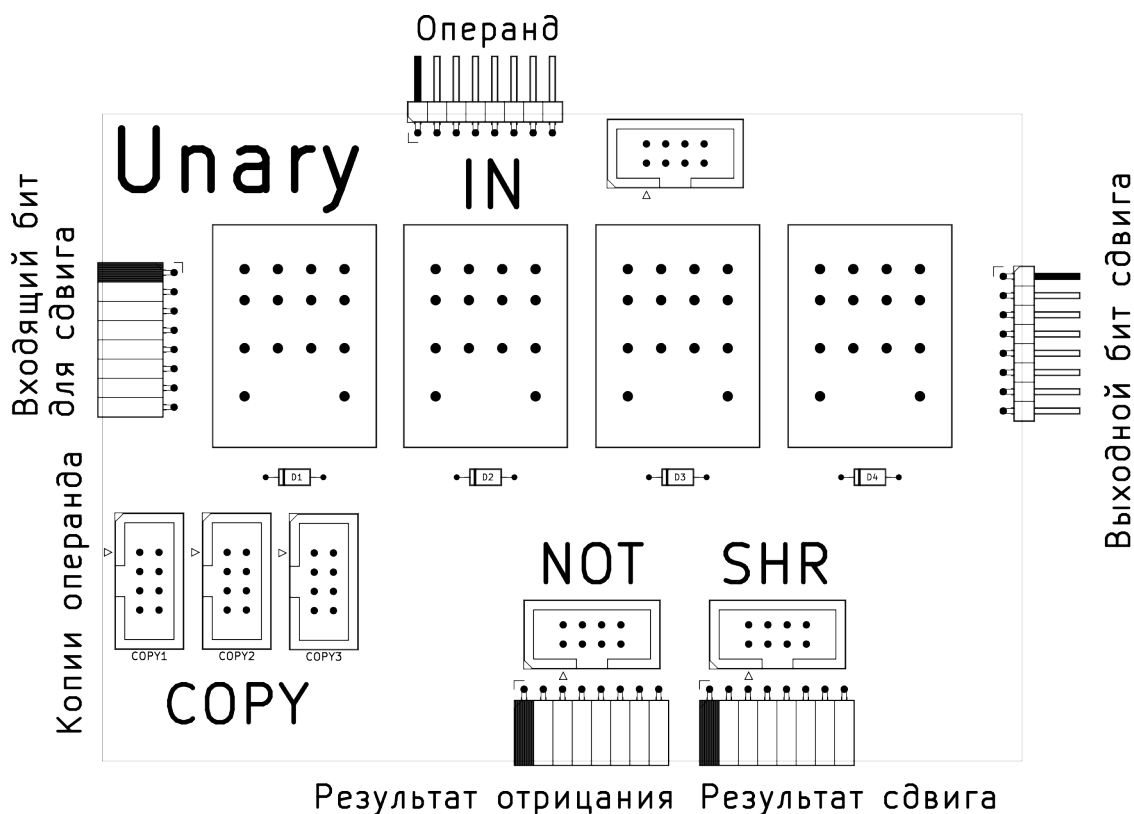


В конструкторе есть готовый модуль только для сдвига вправо. Ещё в вычислениях часто используется сдвиг влево (эквивалентный умножению на два), но его легко реализовать с помощью сложения числа с самим собой.

## 5.4 Модуль унарных логических операций

Модуль для унарных операций выполняет действия над 4-битным числом: сдвиг вправо и инверсия битов. Эти действия выполняются одновременно. То есть с одного выхода можно прочесть сдвинутое число, а с другого инвертированное. И одновременно записать это в разные регистры.

В дополнение к этому с выходных разъёмов модуля можно считать три копии входного числа. Там будут такие же сигналы, но полученные с помощью отдельных цепей.



Модуль для унарных логических операций имеет следующие разъёмы:

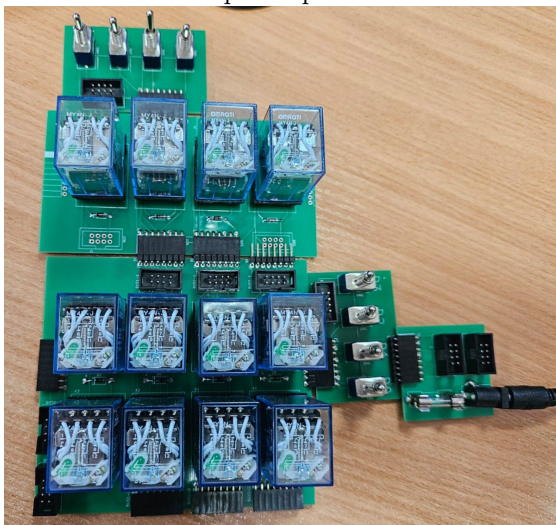
- Слева и справа: шины для каскадирования нескольких модулей. Можно использовать, когда нужен сдвиг чисел больше четырёх бит.
- Сверху: разъёмы для подключения сдвигаемого операнда. Можно подсоединить к какой-нибудь шине, а можно к модулю переключателей или регистру.
- Снизу: два разъёма с выходными сигналами. На один поступает инвертированный операнд, а на другой сдвинутое вправо значение операнда.
- Снизу-слева: три разъёма с копиями операнда.

## 5.5 Практикум

Список модулей:

- Модуль переключателей: 2 штуки
- Модуль унарных операций: 1 штука
- Регистровый модуль: 1 штука

На вход модуля унарных операций подключается модуль с тумблерами. Выходы подключаются к шинам регистра.



1. Отключите все управляющие сигналы.
2. Наберите на тумблерах со входными данными значение 1100.
3. Подключите выходной регистр к шине 1. Убедитесь, что в него записалось значение 0011 (инверсия).
4. Отключите регистр от шины, сбросить его значение.
5. Подключите выходной регистр к шине 2. Убедитесь, что в него записалось значение 0110 (сдвиг вправо).

## 5.6 Задачи

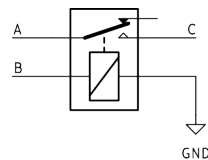
1. Соберите устройство для одновременного вычисления сдвига и инверсии. По сигналу с одного тумблера оба эти результата должны записаться в два разных регистра.
2. Соберите устройство, позволяющее сдвигать содержимое регистра и записывать результат обратно. Чтобы проверить его работу, занесите в регистр значение 1000 и сдвигайте его (потребуется несколько манипуляций) до тех пор, пока не получится 0001. Придётся использовать два регистровых модуля: с одного считывается операнд, а в другой записывается результат. Вторую часть можно выполнять на скорость.

## Глава 6

# Бинарные логические операции

### 6.1 Логическое И

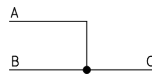
Как вы уже знаете, реле представляет собой управляемый выключатель или переключатель. Если на один контакт  $A$  выключателя подать сигнал, то на другом  $C$  он появится только если на обмотке реле  $B$  есть напряжение (логическая единица).



Получается, что на выходе выключателя сигнал будет только тогда, когда реле включено и на входе выключателя тоже не ноль. Это означает, что такая схема реализует операцию логическое «И»:  $C = A \wedge B$ .

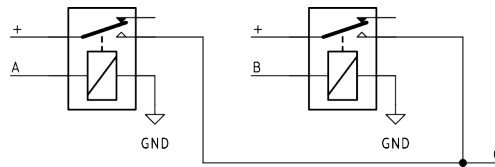
### 6.2 Логическое ИЛИ

Схему для выполнения логического «ИЛИ» можно получить, если соединить два выхода (две цепи). Тогда напряжение на объединённом выходе появится в любом из случаев, когда напряжение подано хотя бы на один из входов  $A$  или  $B$ :  $C = A \vee B$ .



Но у этого способа есть один недостаток. Если на входе  $A$  окажется единица, то так как все входы и выходы соединены в одну цепь, поданное через  $A$  напряжение будет влиять и на вход  $B$ . И когда  $B$  используется ещё где-то, всё может заработать неправильно, например, включится какое-то реле. Такой вот паразитный сигнал —  $A$  влияет на выходы, зависящие от  $B$ .

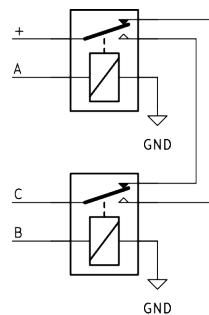
В такой ситуации можно использовать копирование сигнала (которое мы уже видели в унарном модуле) в результате чего получится схема с реле для логического «ИЛИ»:



### 6.3 Исключающее ИЛИ

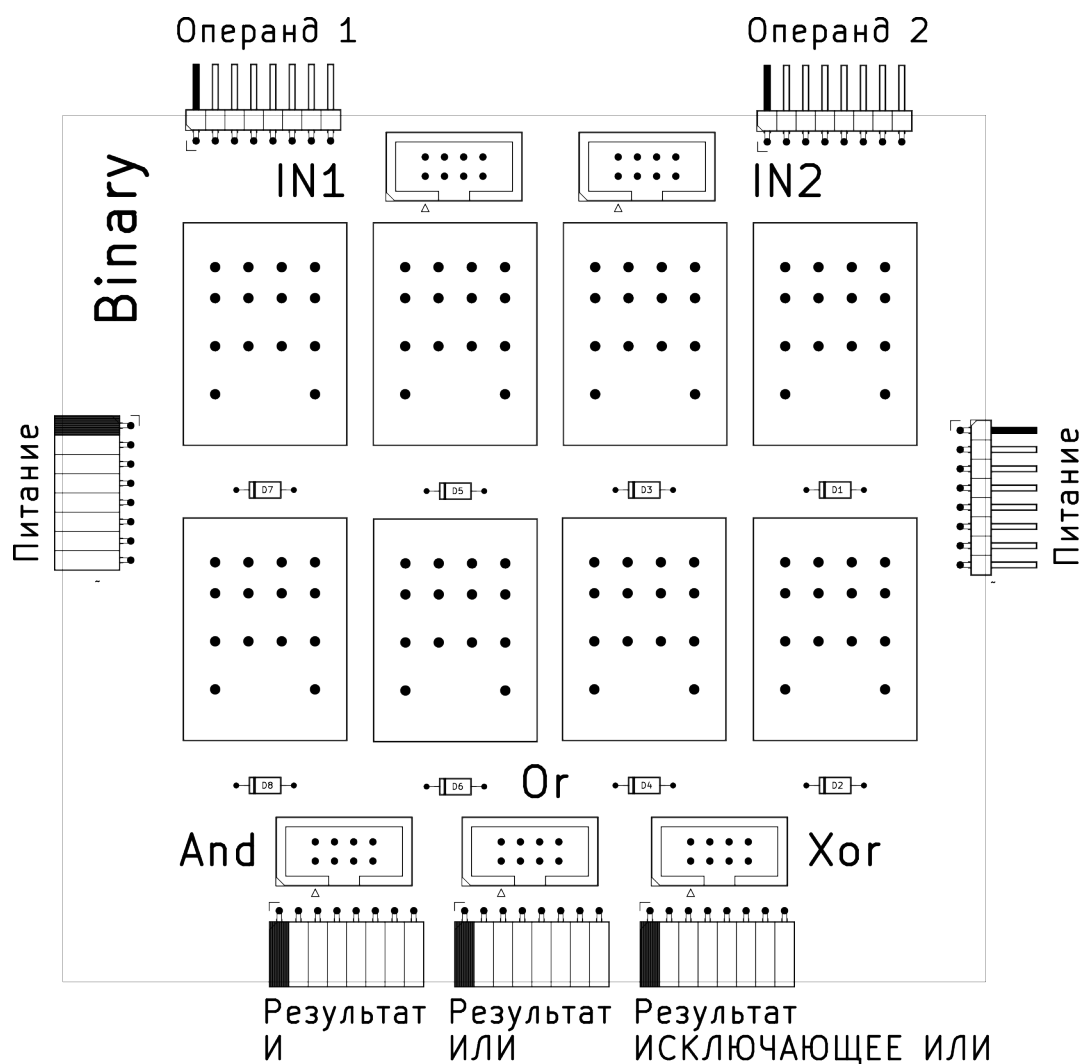
Схема для вычисления «исключающего или» несколько сложнее. Результат операции должен равняться единице только тогда, когда операнды не равны. То есть если на одном из входов уже было напряжение, а потом оно появляется и на другом входе, выход из состояния единицы должен переключиться в ноль.

То же самое должно происходить, если сигналы меняются в другом порядке или одновременно. Такую логику проще всего реализовать с помощью двух переключающих контактов:



Благодаря перекрёстному соединению переключателей проводниками, сигнал на выходе *C* появляется только тогда, когда переключатели на реле находятся в противоположных положениях.

## 6.4 Модуль бинарных логических операций



Модуль логических операций поддерживает вычисление «И», «ИЛИ» и «Исключающее ИЛИ». У модуля есть два четырёхбитных входа и три выхода. Каждый выход отвечает за одну операцию.

Модуль имеет следующие разъёмы:

- Слева и справа: шины для каскадирования нескольких модулей. Полезных сигналов через них не передаётся (в отличие от такого же разъёма у модуля унарных операций), но можно их использовать для фиксации нескольких логических модулей вместе, когда нужны вычисления разрядностью больше четырёх бит.
- Сверху: разъёмы для подключения сигналов-операндов. Можно подсоединить к шинам, а можно к модулям переключателей или регистрам.
- Снизу: три разъёма с выходными сигналами: результатами вычисления операций «И», «ИЛИ», «Исключающее ИЛИ».

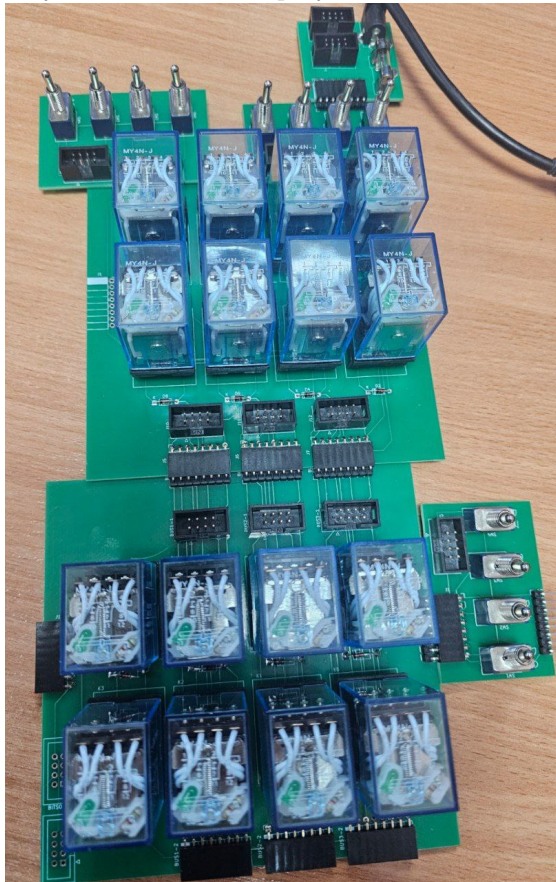
## 6.5 Практикум

### 6.5.1 Работа модуля

Список модулей:

- Модуль переключателей: 3 штуки
- Модуль логических операций: 1 штука
- Регистровый модуль: 1 штука

Ко входам модуля подключаются два модуля с тумблерами, а к выходам — регистр, куда будет записываться результат.



1. Отключите все управляющие сигналы.
2. Наберите на тумблерах первого операнда значение 1100.
3. Наберите на тумблерах второго операнда значение 1010.
4. Подключите выходной регистр к шине 1. Убедитесь, что в него записалось значение 1000 (AND).
5. Отключите регистр от шины, сбросьте его значение.



6. Подключите выходной регистр к шине 2. Убедитесь, что в него записалось значение 1110 (OR).
7. Отключите регистр от шины, сбросьте его значение.
8. Подключите выходной регистр к шине 3. Убедитесь, что в него записалось значение 0110 (XOR).

## 6.6 Задачи

1. Соберите устройство для одновременного вычисления всех трёх логических операций. По сигналу с одного тумблера результаты должны записаться в три разных регистра.

## Глава 7

# Коммутация входов и выходов с помощью кросс-модуля

### 7.1 Кросс-модуль

Иногда на вход модуля нужно подавать биты не в том же порядке, как получены из другого модуля. Например, если потребуется вычислить логическое «И» с третьим битом одного числа и вторым битом другого. А наш логический модуль умеет работать только с определёнными битами, причём сразу со всеми четырьмя.

Чтобы «перемешивать» биты или передавать куда-то только часть сигналов, можно использовать кросс-модуль (он же коммутационная матрица). В нём есть два четырёхбитных входа-выхода (как всегда они работают в обоих направлениях).

Но эти два противоположных разъёма не соединены между собой. Соединения нужно сделать самостоятельно в зависимости от того, какая задача решается. Для примера выше нужно взять два кросс-модуля. В одном бит 0 входа 1 соединить с битом 3 входа 2, а в другом бит 0 входа 1 соединить с битом 2 входа 2.

И теперь, если подключить входы 1 кросс-модулей к логическому модулю, а потом подать сигналы на входы 2, то в нулевых битах выходов логического модуля будут результаты искомого вычисления.

### 7.2 Практикум

Управлять освещением длинного коридора удобно, когда есть несколько выключателей. Например, человеку может потребоваться включить свет, заходя в коридор с одной стороны, а выключить, выходя с другой. Эту задачу можно решить с помощью механических переключателей (у которых один вход и два выхода), но мы попробуем собрать аналогичную логическую схему используя только выключатели (вход один и выход тоже один).

В качестве лампочки будем использовать одно из управляющих реле регистрового модуля. Схема должна позволять в любой момент зажечь или выключить лампочку переключением любого из трёх тумблеров.

Построим таблицу истинности для схемы из выключателей и лампочки: входами  $A$ ,  $B$ ,  $C$  будут положения тумблеров, а выходом  $R$  — состояние лампочки.

Сначала всё выключено, поэтому лампочка тоже не горит:

| A | B | C | R |
|---|---|---|---|
| 0 | 0 | 0 | 0 |

Если один из тумблеров переключить, лампочка загорится:

| A | B | C | R |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 |

Потом можно выключить уже нажатый тумблер (и вернуться к первой строке таблицы), а можно «дойти до конца коридора» и переключить другой. Лампочка должна погаснуть:

| A | B | C | R |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 |

И остаётся вариант, когда переключены все три тумблера. Так как число переключений нечётное, лампочка загорается снова:

| A | B | C | R |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 |

Такая таблица истинности соответствует операции «исключающее или», только для трёх операндов вместо двух. Можете убедиться самостоятельно, что таблица описывает такое выражение:

$$R = A \text{ xor } B \text{ xor } C = (A \text{ xor } B) \text{ xor } C = A \text{ xor } (B \text{ xor } C)$$

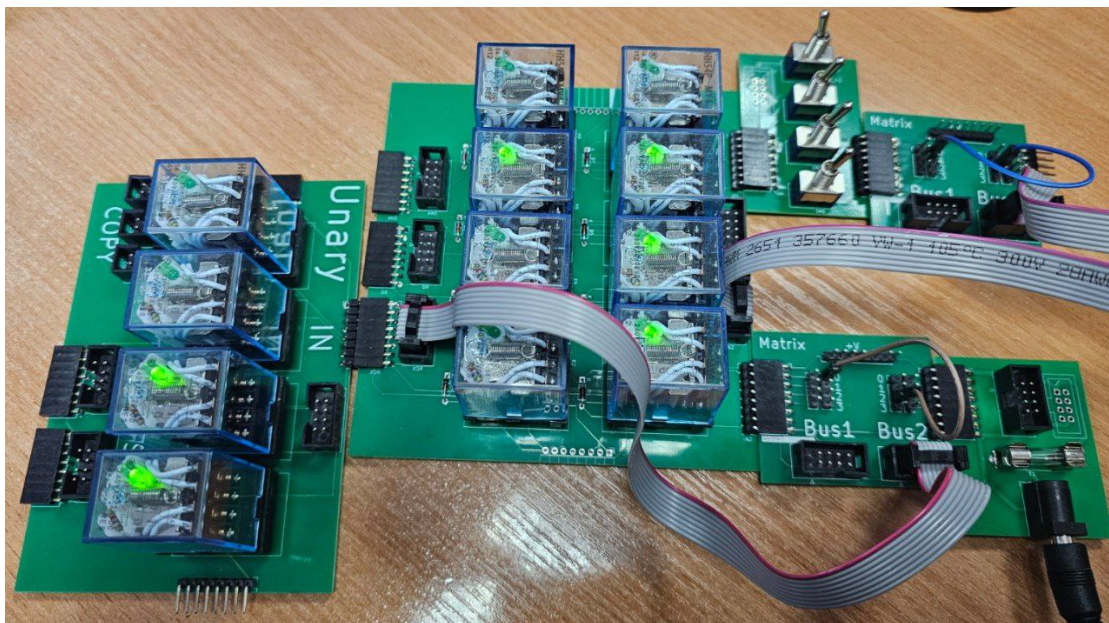
Чтобы его «посчитать» с помощью модулей конструктора нужно немного подумать. Логический модуль считает *xor* для пары четырёхбитных операндов. А у нас тут есть три однобитных. То есть нужно либо взять два логических модуля, либо подавать сигнал с выхода одного однобитного *xor* на вход другого внутри одного и того же модуля.

Первый вариант можно реализовать с помощью такой схемы:



Тут реализуется формула, приведённая выше: сначала выполняется  $A \text{ xor } B$ , а потом с полученным результатом делается  $\text{xor } C$ . Каждый из входных битов расположен на своей плате переключателей. Нужно только выбрать, какой бит использовать, потому что каждый сигнал повторяется 4 раза, по числу задействованных тумблеров. Значение выходного бита показывается с помощью реле на плате модуля унарных операций, подключенного последним в цепочке модулей.

Чтобы использовать только один логический модуль, понадобится пара кросс-модулей для перемещения битов с одного места на другое:



Здесь входные данные задаются с помощью трёх тумблеров: 0 –  $A$ , 1 –  $B$ , 2 –  $C$ . Все сигналы с них подаются на первый вход логического модуля. Но сигнал  $B$  мы тут не используем, а с помощью кросс-модуля перенаправляем его на бит 2 второго входа логического

модуля.

Получается, что бит 2 на выходе будет равен  $C \text{ xor } B$ . С помощью второго кросс-модуля мы передаём выходной сигнал  $\text{xor}$  на нулевой бит второго входа. В результате нулевой бит будет равен  $(C \text{ xor } B) \text{ xor } A$ , а чтобы это увидеть, к выходу  $\text{xor}$  также подключен ещё и модуль унарных операций. Только теперь нельзя выбрать любой бит для работы, а придётся смотреть результат только в нулевом бите. Все вычисления в реальности происходят последовательно, но мы этого не заметим, потому что сигналы распространяются со скоростью света, а реле переключается хоть и помедленнее, но тоже достаточно быстро.

## Глава 8

# Арифметические операции. Сложение

### 8.1 Как складывать двоичные числа

Логическое сложение мы уже видели. Там  $1 + 1 = 1$ , это не подходит для того, чтобы делать обычные арифметические вычисления. Ведь мы хотим получить  $1_2 + 1_2 = 10_2$ .

Посмотрим, как складывать два числа, например 14 и 7. Для этого переведём их в двоичную систему счисления (всё равно ведь компьютеры работают в ней) и запишем их для сложения в столбик:

$$\begin{array}{r} 1 \ 1 \ 1 \ 0 \\ + \ 0 \ 1 \ 1 \ 1 \\ \hline ? \ ? \ ? \ ? \end{array}$$

Сложение в столбик как обычно делается справа налево. Только здесь ещё нужно помнить, что мы работаем в двоичной, а не в десятичной системе счисления. Справа складываются 0 и 1, очевидно, что получается 1:

$$\begin{array}{r} 1 \ 1 \ 1 \ 0 \\ + \ 0 \ 1 \ 1 \ 1 \\ \hline ? \ ? \ ? \ 1 \end{array}$$

Следующая пара цифр это 1 и 1. Мы уже знаем, что должно получиться 10. А это значит, что в текущем разряде остаётся 0, а лишняя единица должна перейти в следующий разряд:

$$\begin{array}{r} 1 \ 1 \ 1 \ 0 \\ + \ 0 \ 1 \ 1 \ 1 \\ \hline ? \ ? + 1 \ 0 \ 1 \end{array}$$

Дальше мы снова складываем 1 и 1, но к ним ещё добавляется перенос из предыдущего разряда. То есть будет  $1 + 1 + 1 = 11$ . Значит в текущем разряде результата теперь единица, а в следующий снова идёт бит переноса.

$$\begin{array}{r} 1 \ 1 \ 1 \ 0 \\ + \ 0 \ 1 \ 1 \ 1 \\ \hline ? + 1 \ 1 \ 0 \ 1 \end{array}$$

Последняя сумма уже знакома:  $1 + 0 + 1 = 10$ . То есть в текущий разряд результата записывается 0, а в следующий идёт 1.

$$\begin{array}{r} 1\ 1\ 1\ 0 \\ +\ 0\ 1\ 1\ 1 \\ \hline 1\ 0\ 1\ 0\ 1 \end{array}$$

Если перевести  $10101_2$  в десятичную систему, то получится  $16 + 4 + 1 = 21$ , как и ожидалось. Теперь нужно придумать, как такую сумму считать с помощью реле.

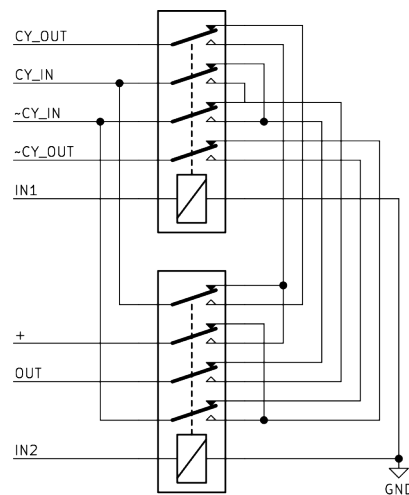
## 8.2 Сумматор

Схему сложения удобнее всего строить из однотипных компонентов, складывающих пару битов. Сумма двух битов может дать результат  $0_2$ ,  $1_2$  или  $10_2$ . То есть при сложении получается уже двухбитовое число: младший бит суммы и старший бит суммы, который называется битом переноса, или битом переполнения, или саггу, или  $CY$ .

Для сложения чисел из нескольких битов, нужно соединить однобитные сумматоры в цепочку и передавать биты переноса от предыдущего сумматора к следующему.

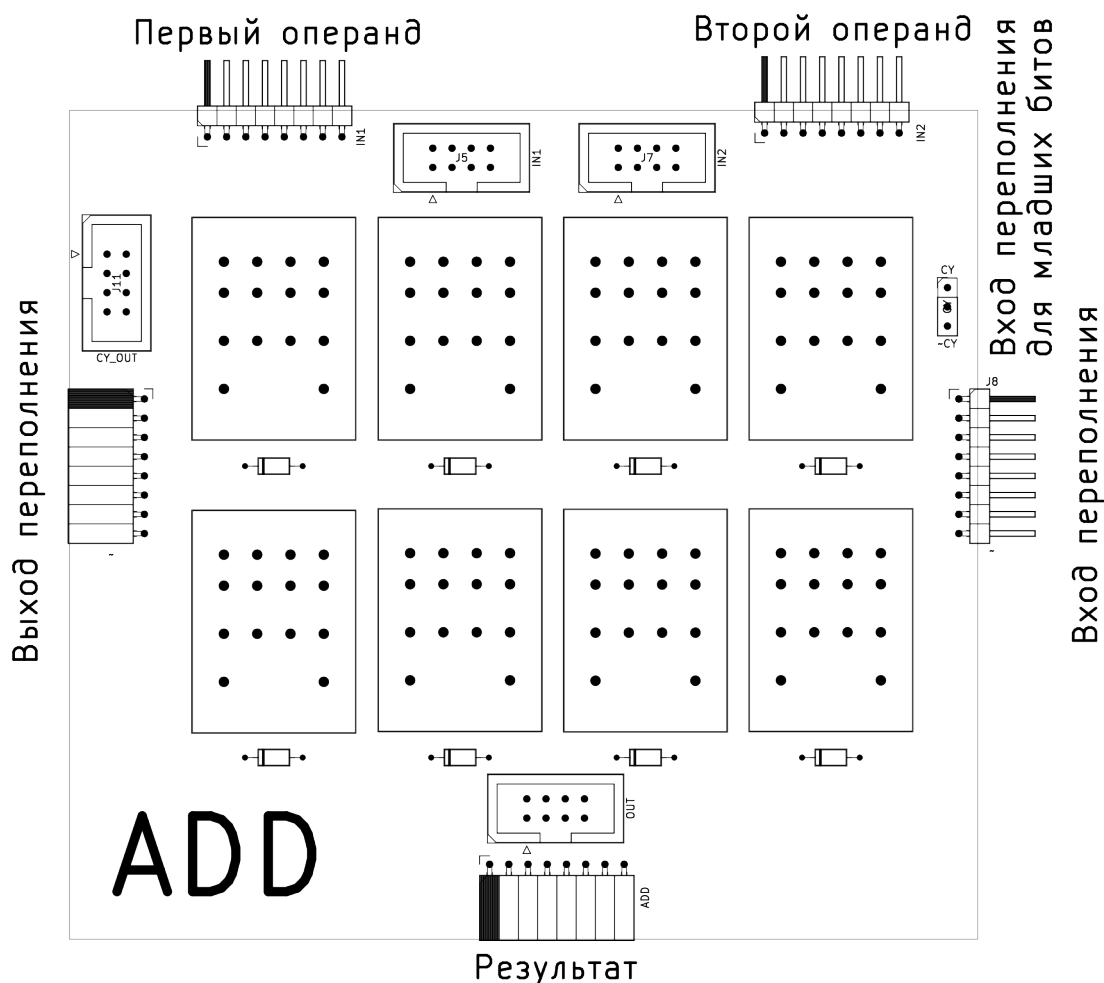
Таким образом, однобитный сумматор получает на вход бит переноса и два бита-слагаемых, а на выходе у него тоже бит переноса и однобитная сумма. С помощью набора таких сумматоров можно сложить числа любой длины.

Особенность реализации сумматора с помощью реле в том, что на входе требуется не только перенос ( $CY$ ), но и инвертированный перенос ( $\sim CY$ ). Такой же инвертированный перенос можно генерировать и на выходе.



Эту схему сумматора изобрёл Конрад Цузе для своих компьютеров в 1940х годах. Несколько таких сумматоров можно соединить последовательно, чтобы складывать многобитные числа. Для этого соединяются соответствующие входы и выходы для переноса и инвертированного переноса, чтобы переполнение переходило из бита 0 в бит 1, из бита 1 в бит 2 и так далее.

### 8.3 Модуль сумматора



Сумматор складывает два четырёхбитных числа, один бит переноса и выдаёт четырёхбитное число и бит переноса.

Модуль имеет следующие разъёмы:

- Слева и справа: шины для каскадирования нескольких модулей. Справа к сумматору приходит сигнал переноса от сумматора младших битов (если он подключен), а помощью шины слева получившийся в результате сложения перенос можно передать на следующий сумматор, записать в регистр или использовать в других логических схемах.
- Рядом с правым разъёмом есть переключатель для включения или выключения бита переноса, если каскадирование не используется. В этом случае переключатель обязательно должна быть в одном из двух положений. Если же справа подключен другой сумматор, то переключатель нужно убрать. Без переключателя (или без замены её тумблерами) не получится, потому что один из двух сигналов (перенос или инвертированный перенос) должен быть единицей.



- Сверху: разъёмы для подключения сигналов-операндов. Можно подсоединить к шинам, а можно к модулям переключателей или регистрам.
- Снизу: разъём, откуда можно считать сумму операндов.

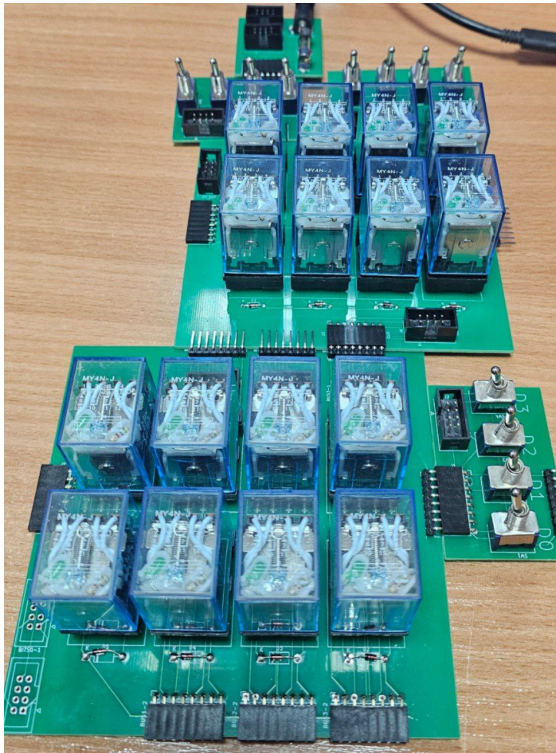
## 8.4 Практикум

### 8.4.1 Работа модуля

Список модулей:

- Модуль переключателей: 3 штуки
- Сумматор: 1 штука
- Регистровый модуль: 1 штука

Ко входам модуля подключаются два модуля с тумблерами, а к выходам — регистр, куда будет записываться результат.



1. Отключите все управляющие сигналы.
2. Установите перемычку для подачи сигнала на  $\sim CY$ .
3. Наберите на тумблерах первого операнда значение 0011 (число 3).
4. Наберите на тумблерах второго операнда значение 1010 (число 10).

5. Подключите выходной регистр к шине 3. Убедитесь, что в него записалось значение 1101 (число 13).
6. Отключите регистр от шины, сбросьте его значение.
7. Установите переключку для подачи сигнала на СУ.
8. Подключите выходной регистр к шине 3. Убедитесь, что в него записалось значение 1110 (число 14).

## 8.5 Задачи

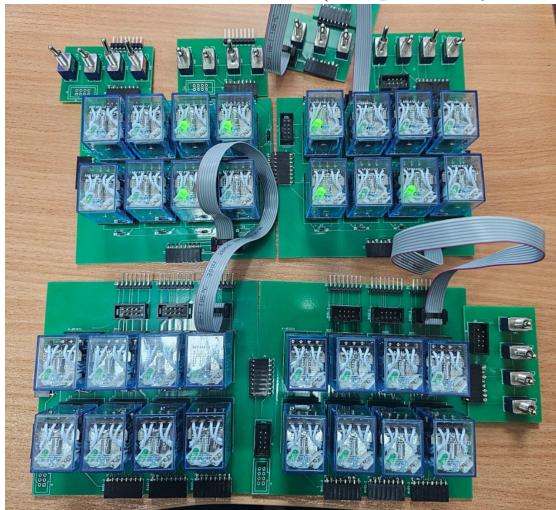
1. Соберите устройство для сложения значения из регистра с константой, набранной на тумблерах. Должна быть предусмотрена запись результата обратно в регистр. Далее управляйте сигналами таким образом, чтобы к регистру последовательно прибавлялась единица, пока он не достигнет значения 1111. Тут необходимо использовать два регистровых модуля: из одного будет считываться операнд для сложения, а во второй записываться результат.

### 8.5.1 Восьмибитный сумматор

Модули сумматоров можно объединять, как и регистры, чтобы работать с данными большей разрядности. Для этого используются боковые разъёмы, через которые передаются сигналы переноса.

### 8.5.2 Практикум

Соедините два регистра и два сумматора. Шина 3 регистров подключается к сумматору. У правого (младшего) сумматора входящий перенос переключкой устанавливается в 0. У левого (старшего) сумматора переключка для переноса убирается, потому что сигнал переноса приходит от младших битов (из правого сумматора).



1. Отключите все управляющие сигналы.

2. Наберите входные данные 00111001 и 00101001.
3. Подключите выход сумматора к регистру тумблером  $D3$ .
4. Проверьте, что в результате получилось число 01100010.

## Глава 9

# Арифметические операции. Вычитание

### 9.1 Вычитание двоичных чисел

Вычитание в компьютерах делается с помощью трюков с переполнением. Что такое  $10 - 7 = 3$  в нашем четырёхбитном вычислителе? Это  $1010 - 0111 = 0011$ . Так как разрядов всего 4, тот же результат можно получить с помощью сложения.

Вспомним, как работает сумматор. На его выходе всегда четырёхбитное число и бит переноса. Если увеличивать 1010, то сначала получится 1011, потом 1100, 1101, 1110, 1111 и наконец 0000.

То есть мы к 10 прибавили 5, получили 15 ( $1010 + 0101 = 1111$ ). После прибавления ещё одной единицы должно было бы получиться 16 (10000), но так как значащих битов всего 4, пятый бит результата пропал (точнее, стал сигналом переноса) и осталось значение 0 (0000).

Остаётся увеличить это число на единицу ещё три раза, чтобы получился правильный ответ 0011. Итого, мы прибавили к 10 число 9 вместо вычитания 5 и получили нужный результат.

Следовательно, вместо вычитания 7 (0111) можно прибавлять 9 (1001), когда речь идёт о четырёхбитных вычислениях.

Такой же фокус можно проделать с любым числом. Быстрый алгоритм (без циклического прибавления единицы, пока не получится как надо) работает так: инвертируем все биты числа, а затем прибавляем единицу.

Такое представление отрицательных чисел называется дополнительным (потому что прибавляли единицу) обратным (потому что инвертировали) кодом.

Число 9 будет дополнительным обратным кодом для 7: из 0111 сначала получится 1000, а потом 1001, что и требовалось. Работает и в обратную сторону — из 1001 инверсией получаем 0110, а после добавления единицы будет 0111.

Таким образом, для вычитания не нужно придумывать новую сложную схему, а достаточно использовать дополнительный обратный код для второго операнда и уже готовую схему для сложения.

## 9.2 Практикум

### 9.2.1 Вычитание через сложение с дополнительным обратным кодом

Список модулей:

- Модуль переключателей: 3 штуки
- Сумматор: 1 штука
- Регистровый модуль: 1 штука

Собрать схему для сложения. Для вычитания вам нужно вычитаемое представлять в дополнительном обратном коде.

1. Отключите все управляющие сигналы.
2. Установите переключку для подачи сигнала на  $\sim CY$  сумматора.
3. Наберите на тумблерах первого операнда значение 1010 (число 10).
4. Наберите на тумблерах второго операнда значение 1101 (число  $-3$ ).
5. Подключите выходной регистр к шине 3. Убедитесь, что в него записалось значение 0111 (число 7).

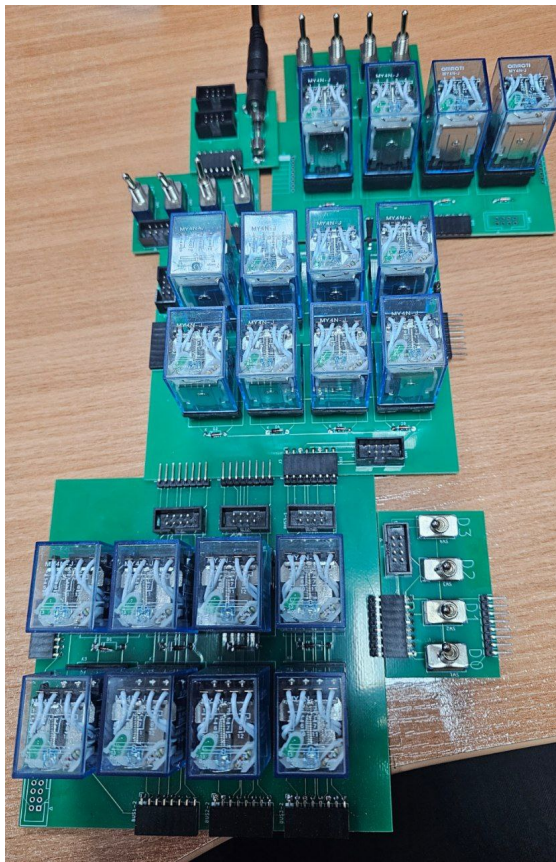
### 9.2.2 Вычитание с помощью сумматора и инвертора

Чтобы вручную не переводить второй операнд, а на втором входе сумматора автоматически получался дополнительный обратный код, сначала нужно преобразовать исходное число с помощью инвертора, а затем прибавить 1, включив вход переноса в сумматоре.

Список модулей:

- Модуль переключателей: 3 штуки
- Модуль унарных операций: 1 штука
- Сумматор: 1 штука
- Регистровый модуль: 1 штука

Собрать схему для сложения, а затем добавить между тумблерами со вторым операндом инвертор (выход NOT блока унарной логики):



1. Отключите все управляющие сигналы.
2. Установите переключку для подачи сигнала на  $CY$  сумматора.
3. Наберите на тумблерах первого операнда значение 1010 (число 10).
4. Наберите на тумблерах второго операнда значение 0011 (число 3).
5. Подключите выходной регистр к шине 3. Убедитесь, что в него записалось значение 0111 (число 7).

## Умножение и деление

В этой главе мы попробуем с помощью сумматора выполнять операцию умножения. Схемы для умножения чисел нужной разрядности получаются намного сложнее схем для сложения, поэтому такого модуля в конструкторе нет.

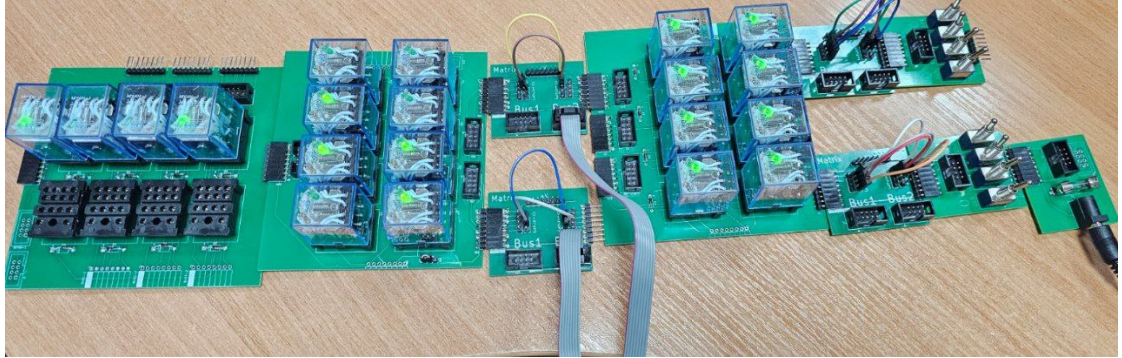
$$\begin{array}{r} \begin{array}{cc} & \begin{array}{c} a_1 \\ b_1 \end{array} \\ \times & \begin{array}{c} a_0 \\ b_0 \end{array} \end{array} \\ \hline \begin{array}{cc} + & \begin{array}{c} a_1 \cdot b_1 \\ a_0 \cdot b_1 \end{array} \end{array} & \begin{array}{c} a_1 \cdot b_0 \\ a_0 \cdot b_0 \end{array} \\ \hline \begin{array}{ccc} a_1 \cdot b_1 & a_0 \cdot b_1 + a_1 \cdot b_0 & a_0 \cdot b_0 \end{array} \end{array}$$
$$\begin{array}{r} \phantom{+} \phantom{1} \phantom{0} \\ \phantom{+} \phantom{1} \phantom{0} \phantom{0} \\ \times \phantom{+} \phantom{1} \phantom{0} \phantom{0} \\ \hline \phantom{+} \phantom{1} \phantom{0} \phantom{0} \\ + \phantom{+} \phantom{1} \phantom{0} \phantom{0} \\ \hline \phantom{+} \phantom{1} \phantom{0} \phantom{0} \end{array}$$
$$a_1a_0 \cdot b_0 = (a_1 \cdot b_0)0 + (a_0 \cdot b_0) = (a_1 \& b_0)0 + (a_0 \& b_0) = a_1a_0 \& b_0b_0$$

Только сначала нужно «размножить» биты операнда  $b$  с помощью коммутационной матрицы.



Выходы с модуля «И» нужно подать на сумматор. Только два бита умноженные на  $b_0$  остаются на месте и подаются на первый вход сумматора, а умноженные на  $b_1$  сдвигаются влево и подаются на второй вход.

Выход сумматора подключается в управляющим входам регистрового модуля, чтобы пронаблюдать результат с помощью светодиодов в реле. Получается такая схема:



Попробуйте набирать с помощью тумблеров разные значения для обоих операндов (конечно же, они должны быть от 0 до 3) и проверьте, правильный ли получается результат.

## 10.2 Калькулятор для пошагового деления

Деление одного целого числа на другое будет простым только в случае, если второе число это степень двойки. Например, при делении на  $100_2$  мы просто отбрасываем две младшие двоичные цифры делимого.

Если же делитель не равен степени двойки, схема сильно усложняется. Поэтому в первых микропроцессорах инструкции деления не существовало и программистам приходилось писать для таких вычислений отдельную подпрограмму.

## 10.3 Задачи

1. Соберите схему для вычитания двух чисел из соответствующей главы.
2. Выполните деление числа 14 на 3 с помощью последовательности вычитаний. Записывайте промежуточные результаты на бумаге.
3. Сделайте то же самое, только добавьте к схеме два регистра, чтобы результат очередного вычитания снова подать на вход сумматора.



## Глава 11

# Калькулятор

### 11.1 Калькулятор для 7 операций

Мы уже рассмотрели все арифметические и логические операции, которые есть в большинстве процессоров (а также будут в нашем релейном компьютере). Но для каждой операции мы собирали свою схему. А ведь компьютеры так не делают, они умеют выбрать любую из доступных операций и вычислить только её.

Следовательно, надо как-то соединить все вычислительные модули в одну схему, чтобы начать работу над процессором. Сделаем несложный калькулятор, в котором результат каждой из операций можно записать в выходной регистр. При этом схему пересобирать не нужно.

Калькулятор будет выполнять следующие операции:

1. Инверсия
2. Сдвиг вправо
3. Побитовое И
4. Побитовое ИЛИ
5. Исключающее ИЛИ
6. Сложение
7. Вычитание

Для ввода значений операндов можно использовать два модуля с тумблерами, подключив их сразу ко всем модулям с помощью шины. Либо же к каждому модулю можно подключить свой набор тумблеров и вводить операнды непосредственно рядом с нужным модулем.

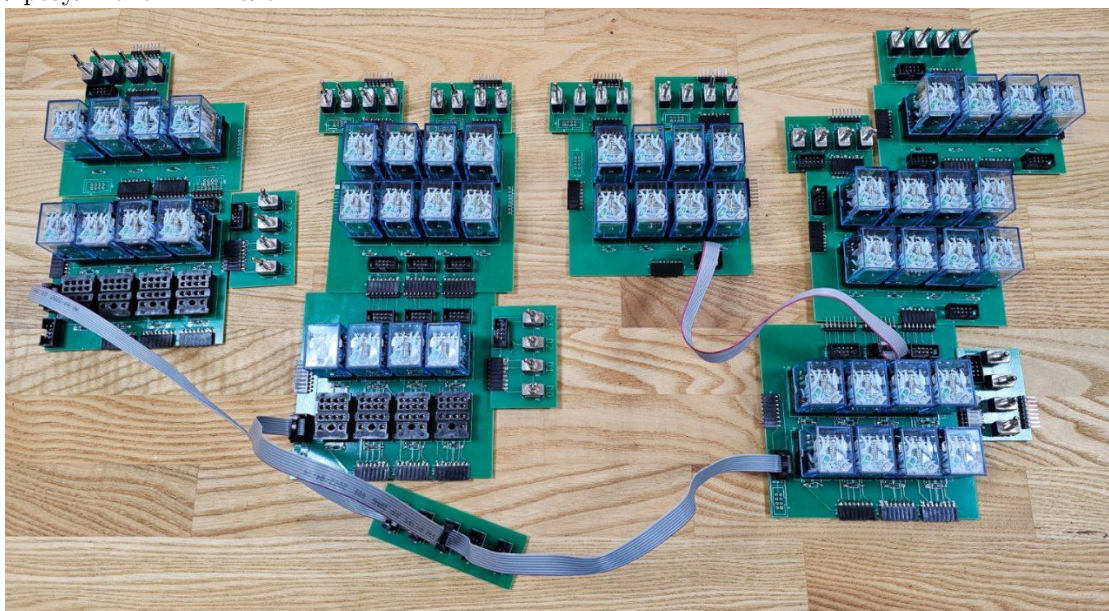
### 11.2 Практикум

Список модулей:

- Модуль переключателей: 10 штук
- Модуль унарных операций: 2 штуки

- Сумматор: 2 штуки
- Модуль логических операций: 1 штука
- Регистровый модуль как шинный формирователь: 2 штуки
- Регистровый модуль: 1 штука
- Модуль шины: 1 штука
- Соединительные шлейфы: 4 штуки

Уже проверенные модули для разных операций соединяются вместе. Для этого сумматоры подключаются к выходному регистру напрямую через его же шинный формирователь, а для выходов логических модулей используются шинные формирователи на платах других регистровых модулей. На этих платах не установлены реле для хранения битов. Вместо этого шины подключаются к первому регистровому модулю, у которого реле для хранения данных установлены. С помощью такого приёма в этот регистр можно записывать любой из 7 результатов вычислений.



Любую из операций можно выполнить так:

1. Отключить все управляющие сигналы.
2. Набрать входные данные для нужной операции.
3. Подключить выход нужного модуля к регистру тумблером.
4. Наблюдать результаты вычислений.
5. Сбросить значение регистра.

Проверьте, что все операции работают корректно и не мешают друг другу (если правильно подавать сигналы для выборки регистров, конечно же).

## Глава 12

### Ответы на задачи

## Часть II

# Строим релейный компьютер

## Глава 13

# Элементы компьютера

### 13.1 Чем компьютер отличается от калькулятора?



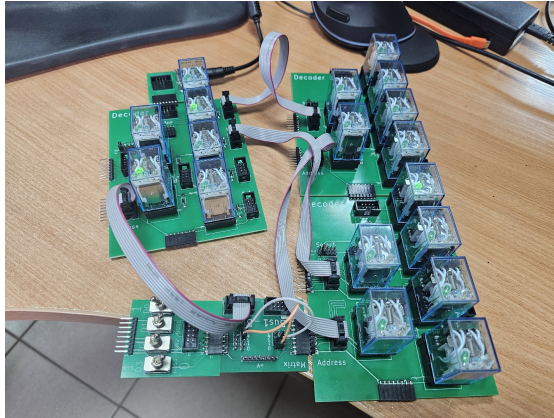
[illegible]

## 14.1 Практикум

### 14.1.1 Устройство, угадывающее животных

### 14.1.2 Ещё больше животных

Пять дешифраторов для каскадирования



### 14.1.3 Выбор одного из четырёх значений с помощью дешифратора

### 14.1.4 Задачи

- Найдите животных, которыми можно было бы заполнить недостающие строки в таблице.



## Глава 15

# Калькулятор с выбором операции

Калькулятор, который мы собирали раньше, поддерживал 7 операций:

1. Инверсия
2. Сдвиг вправо
3. Побитовое И
4. Побитовое ИЛИ
5. Исключающее ИЛИ
6. Сложение
7. Вычитание

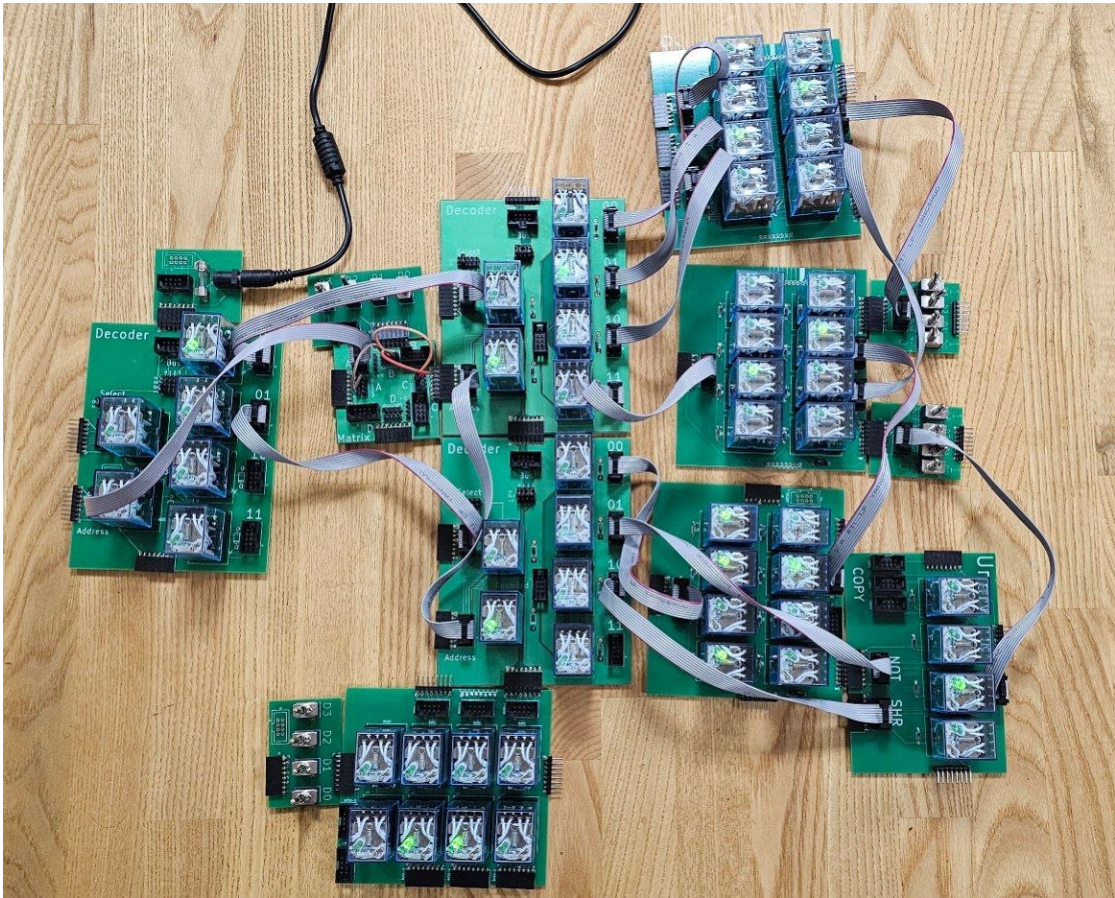
Однако у каждой операции был собственный набор переключателей-операндов. А для логических операций ещё нужно было выбрать нужный выход.

Было бы удобнее отдельно вводить значения операндов, а лишь потом указывать выполняемую операцию.

В итоге для каждого вычисления нужна следующая последовательность действий:

1. Набрать значение первого операнда.
2. Набрать значение второго операнда (если нужна бинарная операция).
3. Выбрать выполняемую операцию.
4. Загрузить получившийся результат в регистр.

## 15.1 Практикум



Можно сделать два варианта вычитания: с использованием отдельного инвертора или с использованием инвертора, который делает унарные операции. Во втором случае унарные операции будут работать со вторым операндом, а не с первым.

Список модулей:

- Модуль переключателей: 4 штуки
- Модуль унарных операций: 1 штука (или 2 штуки)
- Сумматор: 2 штуки
- Модуль логических операций: 1 штука
- Дешифратор: 3 штуки
- Регистровый модуль: 1 штука
- Кросс-модуль: 1 штука

## Глава 16

# Регистровый файл

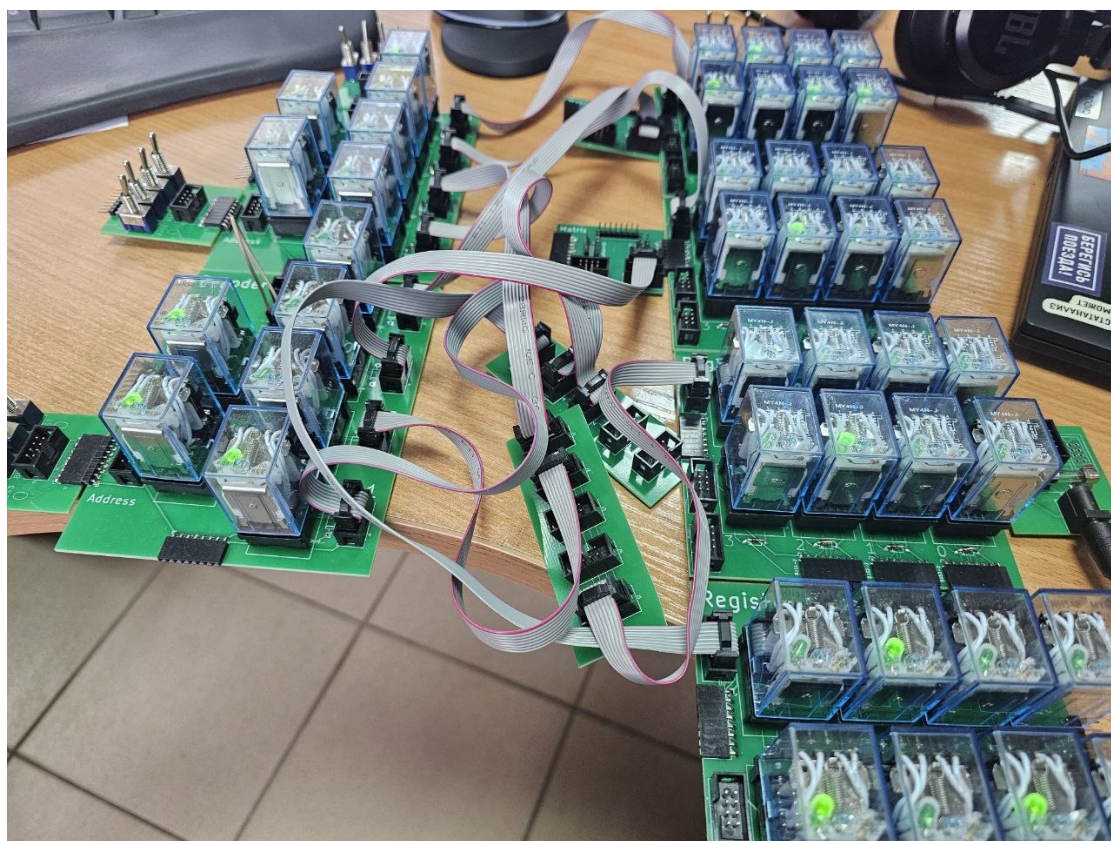
### 16.1 Практикум

Список модулей:

- Регистровый модуль: 4 штуки
- Дешифратор: 2 штуки
- Модуль переключателей: 4 штуки
- Модуль шины: 4 штуки
- Соединительные шлейфы: 8 штук

Дешифраторы соединяются между собой, чтобы сэкономить на коммутируемых управляющих сигналах: вместо двух модулей с переключателями используется один. Первые два бита (сброс и выборка на шину 1) распределяются первым дешифратором, а третий бит (который тоже будет отвечать за выборку на шину 1, но уже другого регистра) — вторым дешифратором. Для этого у дешифраторов соответствующим образом выставляются перемычки.

Каждый выход дешифраторов подключается к управляющему входу соответствующего регистра. Так как вход с разъёмом для шлейфа только один, необходимо использовать модули шин или коммутационные матрицы для подключения выходов с двух дешифраторов к одному входу каждого регистра.



Проверьте работу собранного регистрового файла:

- Установите на входе любого из дешифраторов адрес 0010 и выберите регистр на шину с помощью управляющих тумблеров.
- Установите на тумблерах, подключённых к шине 1 значение 0110. Убедитесь, что оно записалось в третий по счёту регистр (его адрес как раз и будет 0010, так как нумерация начинается с нуля).
- Сбросьте все тумблеры в 0.
- Установите на входе одного дешифратора адрес 0010, а на входе другого — 0011.
- Теперь включите два управляющих бита для выборки регистров (1 и 2). Убедитесь, что третий и четвёртый по счёту регистры подключились к шине 1. При этом в четвёртый регистр скопируется значение из третьего.
- Сбросьте все тумблеры в 0.
- Прodelайте аналогичные действия, чтобы сбросить значения регистров. Для этого нужно будет активировать бит 0 на управляющих тумблерах.

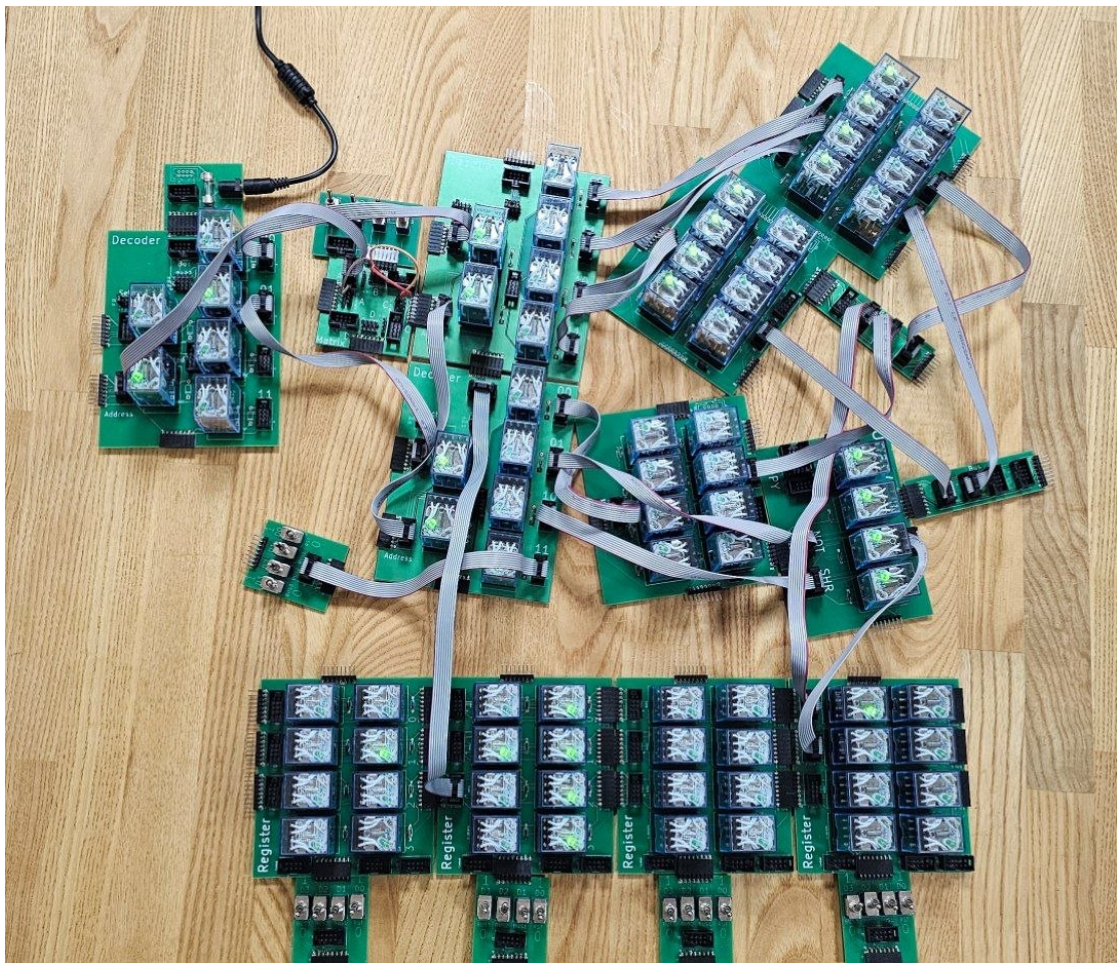
## Глава 17

# АЛУ

Арифметико-логическое устройство (АЛУ) компьютера отвечает за вычисления. В современных компьютерах АЛУ находится внутри микропроцессора. Чтобы что-то посчитать, процессор должен подключить ко входам АЛУ операнды (из регистров или, к примеру, из выполняемой команды). Результат затем помещается ещё в какой-то регистр.



## 17.1 Практикум

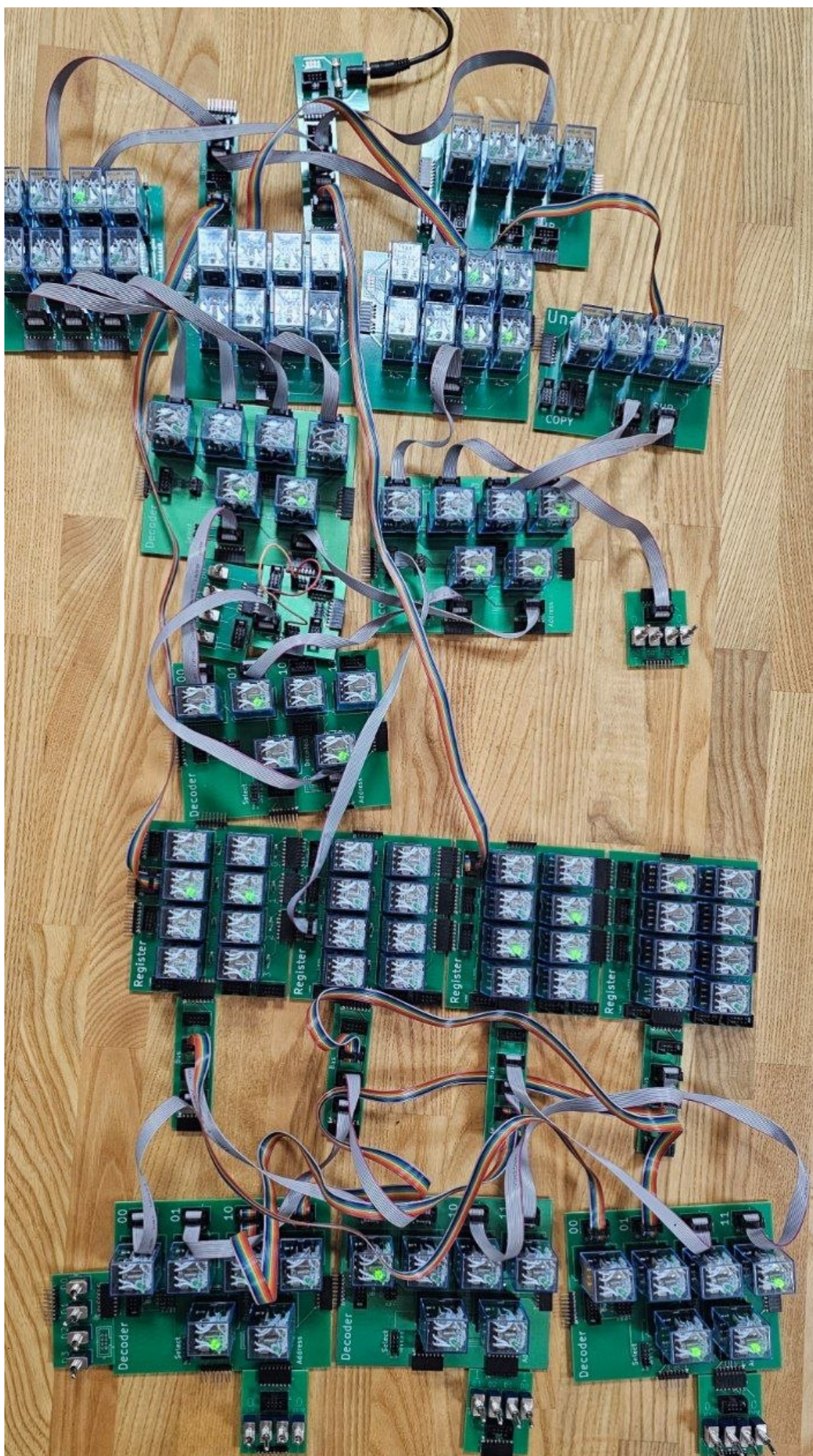


Калькулятор с выбором операндов из регистрового файла.

Список модулей:

- Модуль переключателей: 6 штук
- Модуль унарных операций: 1 штука (или 2 штуки)
- Сумматор: 2 штуки
- Модуль логических операций: 1 штука
- Дешифратор: 3 штуки
- Регистровый модуль: 4 штуки
- Кросс-модуль: 1 штука
- Модуль шины: 2 штуки





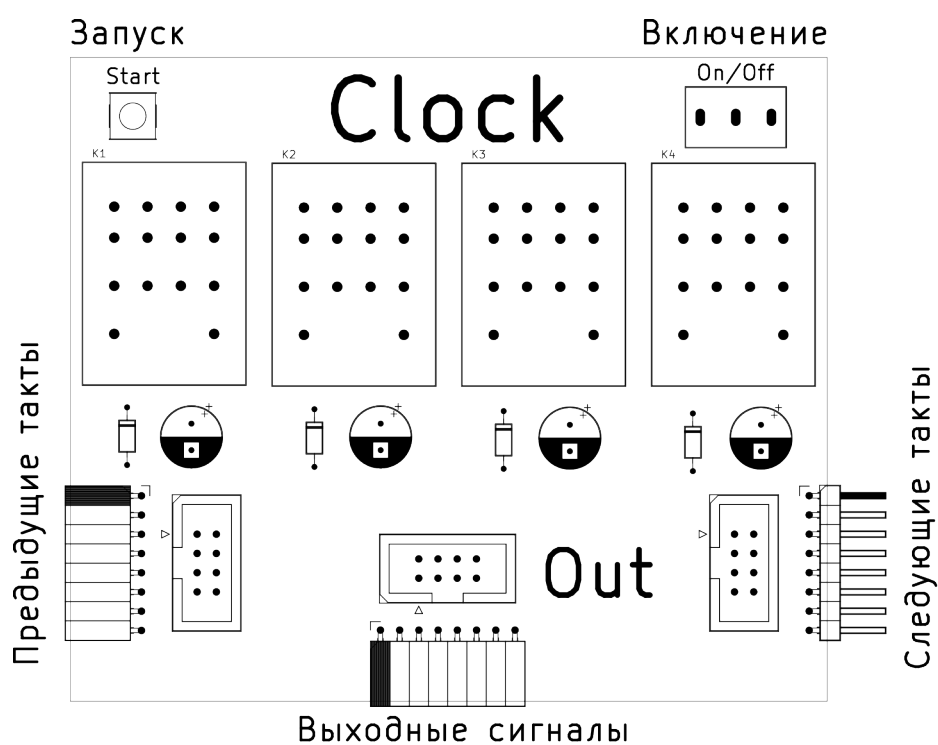
Калькулятор с выбором операндов из регистрового файла с помощью дешифраторов.  
Список модулей:

- Модуль переключателей: 6 штук
- Модуль унарных операций: 1 штука (или 2 штуки)
- Сумматор: 2 штуки
- Модуль логических операций: 1 штука
- Дешифратор: 6 штук
- Регистровый модуль: 4 штуки
- Кросс-модуль: 1 штука
- Модуль шины: 6 штук



## Глава 18

# Тактовый генератор



18.1 Бесконечная последовательность сигналов

18.2 Копирование значения в цикле из одного регистра в другой

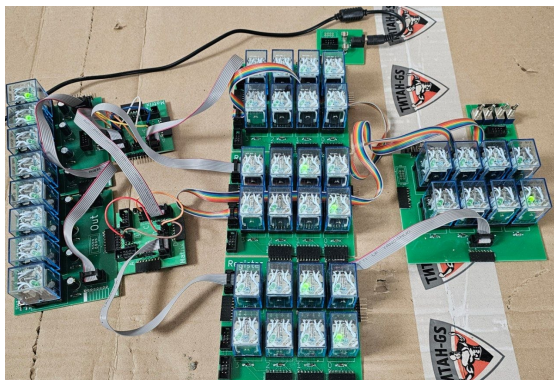
18.3 Автомат для вычисления чисел Фибоначчи

b = 1 (делается с помощью тумблеров)

```
repeat c = a + b a = b b = c
```

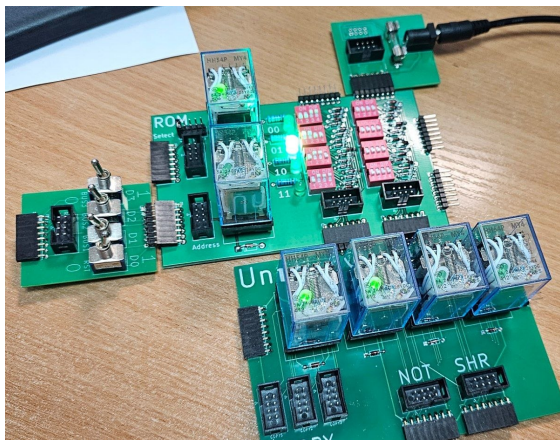
Циклы

1 сбросить c 2 загрузить c = a + b 3 сбросить a 4 загрузить a = b 5 сбросить b 6 загрузить b = c



## Глава 19

### ПЗУ



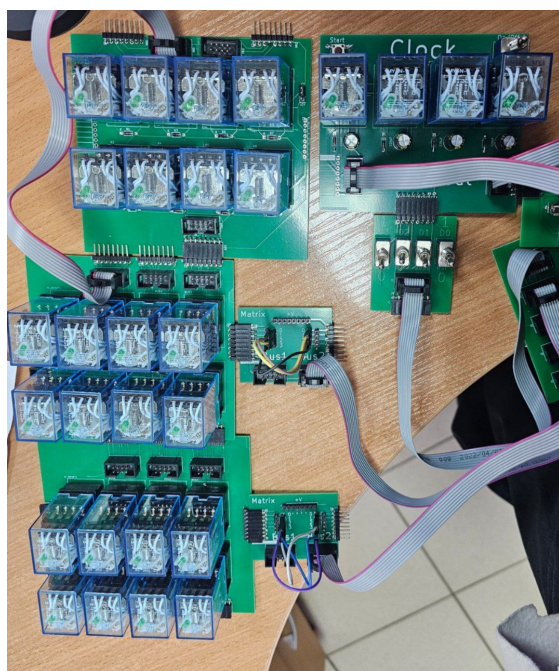
## Глава 20

# Счётчик команд

Счётчик команд — это регистр, который хранит номер выполняемой процессором команды. Сами команды хранятся в памяти. Процессор выполнив одну, переходит к следующей. Следовательно, счётчик команд нужно каждый раз увеличивать на единицу.

### 20.1 Практикум

### 20.2 Циклический инкремент регистра



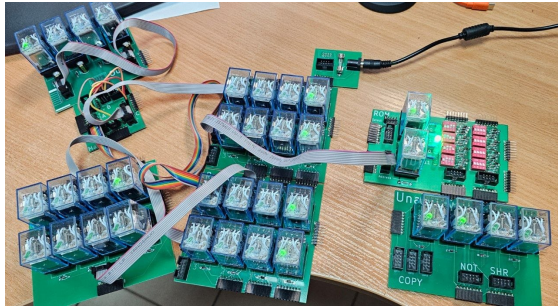
Список модулей:

- Модуль переключателей: .. штук
- Сумматор: 1 штука

### 20.3. ЦИКЛИЧЕСКИЙ ИНКРЕМЕНТ РЕГИСТРА И ВЫБОРКА КОМАНДЫ ИЗ ПЗУ<sup>67</sup>

- Регистровый модуль: 2 штуки

### 20.3 Циклический инкремент регистра и выборка команды из ПЗУ



### 20.4 Загрузка команды из ПЗУ в регистр

## Глава 21

# Набор инструкций

## Глава 22

# Декодер инструкций

Регистры PC - счётчик инструкций T - временный выходной регистр АЛУ I - загруженная инструкция

Выборка инструкции 1 подключить PC сбросить I 2 подключить ROM загрузить I сбросить T 3 подключить PC к сумматору загрузить T 4 сбросить PC 5 скопировать T->PC

Декодирование инструкции АЛУ 1 подключить операнды к АЛУ 2 записать T 3 сбросить целевой регистр 4 загрузить целевой регистр

## Глава 23

# Компьютер



Часть III

Приложения

## Приложение А

### Как самостоятельно собирать модули

## Приложение В

### Список деталей для полного набора